

BSCS FINAL PROJECT

Smart Electricity Bill Prediction System (SEBPS)



Advisor: **Mr. Numan Aslam**

Presented by:
Group ID: F25CS132

Student Reg#	Student Name
L1F22BSCS0501	Ali Abdul Jabbar
L1F22BSCS0513	Shaigan Haider
L1F22BSCS0521	Mohammad
L1F22BSCS0504	Burhan Anwar

Faculty of Information Technology

University of Central Punjab

Smart Electricity Bill Prediction System (SEBPS)

By

Ali Abdul Jabar
Shaigan Haider
Mohammad
Burhan Anwar

Thesis/Project submitted to
Faculty of Information Technology,
University of Central Punjab,
Lahore, Pakistan.

in partial fulfillment of the requirements for the degree of

**BACHELOR OF SCIENCE
IN
COMPUTER SCIENCE**

Abstract

Electricity bills in Pakistan are hard to predict. Tariff slabs change, consumption swings with the season, and extra surcharges keep getting added on top, so most households end up guessing their next bill from the last one. The online portals run by the distribution companies only show past bills and offer no forecasting, no live tracking, and no help with staying inside a budget.

The Smart Electricity Bill Prediction System (SEBPS) was built to close that gap. It is an AI and IoT based system that predicts the monthly electricity bill and helps users keep their usage under control. The completed system reads consumer details from an uploaded bill using Optical Character Recognition (OCR), pulls the past billing history for that consumer, and runs machine learning models to estimate the coming month's units and cost. A small hardware unit built around an ESP32 microcontroller and a PZEM-004 energy meter measures real electricity usage and sends it to the server every few seconds, which improves the prediction and makes the user aware of what they are spending as it happens.

Everything is shown on a web dashboard with a matching mobile view, where users can follow usage trends, the predicted bill, budget progress, and appliance level saving tips. The system also includes an AI chat-bot tuned to answer electricity related questions, which can draw on the user's live unit data, so a user can ask something like how much they have used today or why their bill is trending high and get a grounded, relevant answer. The project brought together several areas of work: regression and machine learning modelling, OCR and image processing, embedded systems and IoT communication, web and backend development, and the slab-based tariff structure used in Pakistan. By the end of the project, SEBPS was able to produce bill estimates that stayed within an acceptable error range of the real bills, track live consumption, and warn users before they crossed their set budget.

Dedication

In the name of **Allah**, the Most Gracious and the Most Merciful.

Every achievement begins with the blessings and guidance of **Allah Almighty**, whose mercy, wisdom, and countless favors have illuminated our path throughout this academic journey. It is only through His endless grace that we were blessed with the strength, patience, determination, and perseverance to overcome every obstacle and successfully accomplish this project. We are forever grateful for His countless blessings, which enabled us to transform our knowledge, ideas, and efforts into a meaningful and practical contribution to the field of intelligent energy management.

We lovingly dedicate this project to our parents, whose unconditional love, endless prayers, unwavering encouragement, and immeasurable sacrifices have been the greatest source of inspiration throughout our lives. They have always believed in our abilities, motivated us during difficult times, and provided every possible opportunity for us to pursue higher education. Their constant support and trust have played an irreplaceable role in shaping both our academic achievements and personal character. This accomplishment is as much theirs as it is ours.

We also dedicate this work to our families, whose patience, understanding, and continuous encouragement gave us the confidence to remain committed to our goals. Their emotional support and belief in our potential inspired us to stay focused despite the many technical and academic challenges encountered during the development of this project.

This project is further dedicated to our respected **project advisor: Mr. Numan Aslam**, faculty members, and the University of Central Punjab, whose knowledge, guidance, constructive criticism, and mentorship significantly contributed to the successful completion of this work. Their commitment to academic excellence encouraged us to think critically, solve problems creatively, and continuously raise the standard of our technical and professional skills. The valuable lessons learned throughout this journey will continue to guide us in our future careers.

Finally, we dedicate **SEBPS** to every household in Pakistan that has ever struggled to anticipate, understand, or manage an electricity bill. Electricity is a necessity, yet the unpredictability of monthly billing places a real and recurring burden on ordinary families. Through this project, we hope to contribute towards relieving that burden by providing an intelligent, accessible, and locally adapted platform that empowers consumers to take control of their energy usage and plan their expenses with confidence. It is our sincere hope that SEBPS will inspire future innovations in smart energy management and make a meaningful positive difference in the lives of its users.

Acknowledgements

All praise and gratitude are due to **Allah Almighty**, the Most Gracious and the Most Merciful, whose countless blessings, guidance, and mercy enabled us to complete this project successfully. His wisdom and strength gave us perseverance to overcome every challenge encountered throughout this journey. We would like to express our heartfelt gratitude to our respected Project Advisor for his invaluable guidance, continuous encouragement, constructive criticism, and unwavering support throughout the development of SEBPS. His insightful suggestions, technical expertise, and constant motivation played a significant role in improving the quality of our work and in successfully completing this project.

We are also sincerely thankful to all the faculty members of the Faculty of Information Technology and Computer Science, University of Central Punjab, for providing us with the knowledge, skills, and academic environment necessary to undertake and accomplish this project. Their dedication to excellence has greatly contributed to our academic and professional development. We extend our deepest appreciation to our parents and families for their unconditional love, endless prayers, patience, encouragement, and sacrifices. Their constant support and belief in our abilities motivated us to remain determined and focused throughout our academic journey, particularly during the more demanding phases of hardware integration, machine learning development, and system testing. We are additionally grateful to the open-source communities behind EasyOCR, scikit-learn, TensorFlow, Flutter, Node.js, and the ESP32 Arduino ecosystem, whose freely available tools and documentation made the technical ambitions of this project achievable within an academic timeline.

Finally, we are grateful to everyone who directly or indirectly contributed to the successful completion of SEBPS. We sincerely hope that this project serves as a meaningful contribution towards smarter household energy management in Pakistan and provides lasting value to its future users.

List of Figures

Figure 1: Use-Case Diagram.....	11
Figure 2: Entity-Relationship Diagram.....	16
Figure 3: Use- Case Diagram.....	17
Figure 4: Class Diagram	18
Figure 5: Sequence (Get Bill Prediction).....	19
Figure 6: Sequence (Manage Budget Alerts).....	19
Figure 7: Sequence (Process Electricity Bills).....	20
Figure 8: DFD (Data Flow Diagram).....	20
Figure 9: Architecture Diagram.....	25
Figure 10: WorkFlow Diagram.....	45

List of Tables

Table 1: UC-1	12
Table 2: UC-2	13
Table 3: UC-3	14
Table 4: UC-4	14
Table 5: UC-5	15
Table 6: UC-6	16
Table 7: Primary REST API Endpoints	28
Table 8: Data Reused	29
Table 9: Libraries Used.....	29
Table 10: SEBPS Full Technology Stack	30
Table 11: Architecture Evaluation Against Quality Attributes.....	32
Table 12: TC-1: Upload and Extract Bill Data	56
Table 13: TC-2: Fetch Historical Billing Data	57
Table 14: TC-3: Predict Future Electricity Bill	58
Table 15:TC-4: Monitor Real-Time Consumption	59
Table 16: TC-5: Set Budget and Receive Alerts.....	60
Table 17: TC-6: Get Appliance-Level Recommendations	60
Table 18: Summary of All Test Results.....	61
Table 19: Project Completion Status	62
Table 20: Objective(s)/Target(s) Status	63
Table B.21: List of non-trivial defects.....	65

Table of Contents

Abstract.....	i
Dedication	ii
Acknowledgements	iii
List of Figures.....	iv
List of Tables	iv
Table of Contents	v
Revision History	vii
Chapter 1. Introduction.....	1
1.1 Product (Problem Statement).....	1
1.2 Background.....	1
1.3 Objective(s)/Aim(s)/Target(s)	2
1.4 Scope	2
1.5 Business Goals.....	2
1.6 Challenges	3
1.7 Learning Outcomes	3
1.8 Nature of End Product	4
1.9 Related Work/ Literature Survey/ Literature Review.....	4
1.10 Document Conventions	5
1.11 Miscellaneous	5
Chapter 2. Overall Description	7
2.1 Product Features	7
2.2 Functional Description	7
2.3 User Classes and Characteristics	8
2.4 Design and Implementation Constraints.....	9
2.5 Assumptions and Dependencies	10
Chapter 3. System Requirements	11
3.1 Functional Requirements.....	11
3.1.1 Upload and Extract Bill Data	12
3.1.2 Fetch Historical Billing Data.....	12
3.1.3 Predict Future Electricity Bill	13
3.1.4 Monitor Real-Time Consumption	14
3.1.5 Set Budget and Receive Alerts.....	14
3.1.6 Get Appliance-Level Recommendations.....	15
3.1.7 Requirements Analysis and Modeling	16
3.2 Nonfunctional Requirements.....	21
3.2.1 Performance Requirements	21
3.2.2 Safety Requirements	21
3.2.3 Security Requirements	22
3.2.4 Additional Software Quality Attributes	22
Chapter 4. Technical Architecture	24
4.1 Application and Data Architecture	25
4.1.1 Layered Architecture Overview	25
4.1.2 Data Flow Summary.....	25
4.1.3 Database Schema Design	26
4.2 Component Interactions and Collaborations	26
4.2.1 OCR Based Bill Processing Flow.....	26
4.2.2 IoT Real Time Monitoring Flow	26
4.2.3 Machine Learning Bill Prediction Flow	27
4.2.4 Budget Alert Management Flow.....	27
4.2.5 AI Chatbot Flow	27

4.2.6	API Endpoint Catalogue.....	27
4.3	Design Reuse and Design Patterns	28
4.3.1	Applied Design Patterns.....	28
4.3.2	Reused Libraries and Components.....	29
4.4	Technology Architecture	30
4.4.1	Full Technology Stack	30
4.4.2	Frontend Architecture	30
4.4.3	Backend Architecture.....	31
4.4.4	IoT Communication Architecture	31
4.4.5	Deployment Architecture	31
4.5	Architecture Evaluation.....	31
Chapter 5.	Detailed Design and Implementation	35
5.1	Component-Component Interface	35
5.2	Component-External Entities Interface	35
5.3	Component-Human Interface	37
5.4	Screenshots/Prototype	45
5.4.1	Workflow	45
5.4.2	Screens	46
5.5	Additional Information.....	55
5.6	Other Design Details	55
5.6.1	Hardware Interface Design.....	55
5.6.2	Software and Hardware Integration	55
5.6.3	Billing Logic Design.....	55
Chapter 6.	Test Specification and Results	56
6.1	Test Case Specification	56
6.2	Summary of Test Results.....	61
Chapter 7.	Project Completion Status/Conclusion	62
References		63
Appendix A Glossary		64
Appendix B IV & V Report		65
(Independent verification & validation)		65
Appendix C Deployment/Installation Guide		66
Using the Hosted Web App		66
Running the Full Stack Locally with Docker.....		66
Service Overview.....		67
Project Structure.....		67
Appendix D User Manual		68
Getting Started		68
Adding Your Electricity Data		68
Predicting Your Bill.....		69
Live Monitoring with the IoT Meter.....		69
Budgeting and Alerts		69
Appliances and Recommendations		69
The AI Chatbot.....		69
Notifications and Profile		69

Revision History

Name	Date	Reason For Changes	Version

Chapter 1. Introduction

The Smart Electricity Bill Prediction System (SEBPS) was developed to deal with the uncertainty that electricity consumers in Pakistan face every month. Bills move up and down for reasons that are not always clear to an ordinary user, which makes household budgeting difficult. This document is the final design and completion report for SEBPS. It describes the system that was actually built, its architecture, how the components talk to each other, the technologies used, the testing carried out, and where the project stands against the goals set at the start.

SEBPS combines machine learning, automation, and IoT so that users get a bill forecast, live consumption monitoring, AI-Based Recommendations as well as simple budgeting tools in one place. This phase builds on the requirements and design from the earlier phases and reports on the working, completed system. The Smart Electricity Bill Prediction System (SEBPS) was developed to deal with the uncertainty that electricity consumers in Pakistan face every month. Bills move up and down for reasons that are not always clear to an ordinary user, which makes household budgeting difficult. This document is the final design and completion report for SEBPS. It describes the system that was actually built, its architecture, how the components talk to each other, the technologies used, the testing carried out, and where the project stands against the goals set at the start.

SEBPS combines machine learning, automation, and IoT so that users get a bill forecast, live consumption monitoring, and simple budgeting tools in one place. This phase builds on the requirements and design from the earlier phases and reports on the working, completed system

1.1 Product (Problem Statement)

SEBPS is a web and mobile based energy management system supported by a small piece of IoT hardware. It predicts household electricity bills, tracks real time energy use, and helps people manage their monthly electricity spending before the bill arrives instead of after.

The system solves the problem of not knowing what the next bill will look like. It uses OCR to read the consumer information from an uploaded bill, retrieves the past billing records for that consumer, and runs trained machine learning models to forecast the coming month's units and cost. Alongside this, the ESP32 and PZEM-004 unit measures live usage and keeps the system updated with real readings.

In terms of what it is, SEBPS is a software system combined with hardware that applies data analytics, machine learning, and real time monitoring to a real local problem. It is not just an algorithm or a dataset on its own. It is a complete application built around Pakistan's electricity billing structure, with the prediction engine, the OCR pipeline, the IoT meter, the dashboards, and an electricity focused AI chatbot all working together as a single product. The chatbot is scoped to energy and billing topics and is fed the user's live readings and bill context, so its answers stay relevant to that user rather than being generic.

1.2 Background

Cost electricity consumers in Pakistan work out their expected bill by looking at the previous bill or by rough estimation. The official distribution company portals, such as the LESCO bill portal, let users view and download their past bills, but they stop there. There is no forecasting, no live tracking, and

no help with staying inside a budget. On the other side, several international smart energy products focus on real time monitoring and home automation, but they are built around foreign tariff structures and do not understand the slab system, fuel adjustment charges, and taxes that appear on a Pakistani bill.

SEBPS sits between these two. It takes the historical data that the portals already hold, adds OCR so the user does not have to type anything in, layers machine learning forecasting on top, and connects a real energy meter for live readings. Most comparable student projects in this area have done either monitoring on its own or simple prediction on its own. The thing that sets SEBPS apart is that the whole pipeline is tuned to local billing rules, so the predicted bill actually resembles the real one a household receives.

1.3 Objective(s)/Aim(s)/Target(s)

The objectives set for SEBPS were:

- Build an intelligent system that predicts the monthly electricity bill from historical and real time consumption data.
- Use OCR and automation so users do not have to manually enter bill details.
- Train machine learning models that forecast consumption and cost with acceptable accuracy.
- Enable real time energy monitoring through IoT hardware.
- Provide budgeting tools and energy saving recommendations to help users control their expenses.
- Offer an AI chatbot that answers electricity related questions using the user's live unit data and bill history.
- Deliver a secure and easy to use web and mobile interface for household energy management.

By the end of the project these objectives were largely met.

1.4 Scope

SEBPS covers the prediction of electricity bills, monitoring of consumption, and budgeting for household users. This includes OCR based bill scanning, automated retrieval of past records, machine learning forecasting, real time data collection through the IoT unit, and visualization on the web and mobile dashboards.

The project does not handle online bill payment and does not connect to any official payment gateway. It stays on the side of prediction, monitoring, and advice. The focus throughout was the household consumer rather than commercial or industrial connections.

1.5 Business Goals

The goals SEBPS works toward are:

- Cutting down the uncertainty around the monthly electricity bill by giving an early estimate.
- Helping families plan for the expense before it lands.
- Encouraging more careful energy use through live feedback and practical tips.
- Building a base that can later be extended to other distribution companies and smarter energy features.

Making people more aware of where their electricity is actually going.

1.6 Challenges

The development of the Smart Electricity Bill Prediction System (SEBPS) involved several technical and practical challenges. One of the primary challenges was the limited availability of historical electricity consumption data, as LESCO provides only the previous twelve months of billing records. This restricted the amount of data available for prediction and analysis.

Another challenge was incorporating multiple billing components such as electricity tariffs, taxes, fixed charges, and Fuel Price Adjustment (FPA), which frequently change and directly impact the final electricity bill. Ensuring accurate bill estimation while considering these dynamic factors required careful system design.

The integration of hardware components, including the ESP32 microcontroller and the PZEM-004T energy meter, also presented challenges. Establishing reliable communication between the hardware and the software application while maintaining real-time energy monitoring required extensive testing and calibration.

Developing a user-friendly mobile interface capable of displaying real-time consumption, appliance management, budget tracking, and bill predictions in an intuitive manner was another significant challenge. The application needed to remain responsive while processing user inputs and continuously updating energy data.

Finally, ensuring system scalability and maintainability required designing a modular architecture so that future enhancements, such as additional utility providers or advanced forecasting models, could be integrated without major modifications.

1.7 Learning Outcomes

The development of SEBPS provided valuable technical and professional learning experiences throughout the project lifecycle. The major learning outcomes include:

- Understanding the complete Software Development Life Cycle (SDLC) from requirement analysis to deployment.
- Designing and developing a mobile application using modern software engineering principles.
- Integrating IoT hardware (ESP32 and PZEM-004T) with a software application for real-time electricity monitoring.
- Understanding electricity billing structures, including tariff calculations, taxes, fixed charges, and Fuel Price Adjustment (FPA).
- Implementing data collection, preprocessing, and consumption analysis techniques.
- Developing forecasting and bill prediction mechanisms based on historical consumption patterns and user inputs.
- Improving problem-solving, debugging, and system integration skills.
- Gaining practical experience in database management and cloud-based application development.

- Enhancing teamwork, project planning, documentation, and communication skills.
- Learning to design scalable and maintainable software systems suitable for future enhancements.

1.8 Nature of End Product

The Smart Electricity Bill Prediction System (SEBPS) is a web and mobile-based electricity consumption management and bill prediction application integrated with IoT technology. The system enables users to monitor real-time electricity consumption, estimate future electricity bills, manage household appliances, and track monthly energy budgets.

The application collects user inputs regarding appliances, operating hours, electricity tariff information, and real-time energy consumption data from the ESP32 and PZEM-004T energy meter. Using this information, the system calculates estimated electricity costs while considering applicable taxes and Fuel Price Adjustment (FPA).

The final product provides several key functionalities, including:

- User registration and secure login.
- Real-time electricity consumption monitoring.
- Electricity bill prediction.
- Monthly budget planning and tracking.
- Appliance-wise energy consumption management.
- Historical electricity usage analysis.
- Support for multiple electricity meters within a single account.
- Interactive dashboards and graphical reports for better energy insights.

The system is designed to help consumers better understand their electricity usage, reduce unnecessary energy consumption, and improve financial planning by providing accurate electricity bill estimates before the official bill is issued.

1.9 Related Work/ Literature Survey/ Literature Review

Electricity consumption forecasting has become an important research area due to increasing energy demand and the growing adoption of smart grid technologies. Numerous researchers have proposed intelligent systems for predicting electricity consumption using historical usage patterns, weather conditions, occupancy behavior, and appliance-level monitoring.

Many existing studies focus on short-term load forecasting using machine learning algorithms such as Linear Regression, Decision Trees, Random Forest, Support Vector Machines (SVM), and Artificial Neural Networks (ANN). These approaches generally achieve good prediction accuracy when large historical datasets are available.

Recent research has also explored Internet of Things (IoT)-based energy monitoring systems that utilize smart sensors and energy meters to collect real-time electricity consumption data. Devices such as ESP32, Arduino, and PZEM energy meters are widely used due to their affordability, reliability, and ease of integration with mobile and cloud applications.

Several commercial applications provide electricity monitoring and billing information; however, most only display historical consumption after the bill has already been generated. They generally do not allow users to estimate upcoming bills based on appliance usage, budget constraints, or real-time energy consumption.

The proposed Smart Electricity Bill Prediction System (SEBPS) addresses these limitations by combining real-time IoT-based energy monitoring with electricity bill prediction, appliance management, and budget tracking in a single mobile application. Unlike existing systems that rely solely on historical billing information, SEBPS incorporates user-defined appliance usage, tariff calculations, taxes, and Fuel Price Adjustment (FPA) to provide more practical and informative electricity cost estimation.

This integrated approach enables users to make informed decisions regarding energy consumption, helping reduce electricity costs while promoting efficient energy management.

1.10 Document Conventions

To ensure clarity, consistency, and readability across this Software Requirements Specification (SRS), the following conventions are used:

Typography

- **Headings:**
 - *Section Headings (e.g., 1.0, 2.1)* — **Times New Roman, 14 pt, Bold**
 - *Subheadings (e.g., 1.1.1)* — **Times New Roman, 12 pt, Bold**
- **Body Text:**
 - **Arial, 11 pt, Regular**

Emphasis

- **Bold Text** — Highlights important concepts, system names, or key requirements.
- *Italic Text* — Used for external components, tools, files, menus, or systems (e.g., *SQLite Database*, *Unity Engine*).

Numbering

- Follows the IEEE-style hierarchical format:
Section → Subsection → Sub-subsection
Example: 3.1 → 3.1.1

1.11 Miscellaneous

The Smart Electricity Bill Prediction System (SEBPS) has been developed as a practical solution to assist electricity consumers in monitoring their energy consumption and estimating future electricity bills. The system combines real-time energy monitoring with bill prediction, appliance management, and budget tracking to encourage efficient electricity usage.

The application has been designed using a modular architecture, making it easy to maintain, update, and extend in the future. Additional features, such as support for other electricity distribution

companies, cloud-based analytics, smart notifications, and integration with smart home devices, can be incorporated with minimal changes to the existing system.

The project emphasizes usability, reliability, and scalability. A simple and intuitive user interface enables users with limited technical knowledge to operate the application effectively. Security measures such as authenticated user access and secure data storage have been considered to protect user information.

Although the current implementation is developed using the available billing structure and electricity consumption data, the system provides a solid foundation for future enhancements, including more advanced forecasting techniques, personalized energy-saving recommendations, and wider deployment across residential and commercial environments.

Overall, SEBPS demonstrates the practical application of software engineering, mobile application development, and IoT technologies to address real-world energy management challenges while promoting energy conservation and financial awareness among electricity consumers.

Chapter 2. Overall Description

2.1 Product Features

The Smart Electricity Bill Prediction System (SEBPS) is a mobile-based application integrated with IoT technology to help users monitor electricity consumption, predict monthly electricity bills, and manage household energy usage efficiently. The system provides a centralized platform where users can monitor their electricity usage and make informed decisions to reduce electricity costs.

The major features of the system include:

- User Registration and Secure Login
- User Profile Management
- Real-Time Electricity Consumption Monitoring
- Integration with ESP32 and PZEM-004T Energy Meter
- Electricity Bill Prediction
- Monthly Energy Budget Planning
- Appliance Management and Power Rating Configuration
- Historical Electricity Consumption Analysis
- AI-Chatbot trained to answer queries regarding your household data.
- Monthly Consumption Reports and Graphical Visualization
- Notification and Alert System
- LESCO Tariff, Taxes, and Fuel Price Adjustment (FPA) Based Bill Calculation
- Dashboard for Real-Time Energy Insights

The system is designed to provide users with a simple, accurate, and interactive platform for managing electricity consumption.

2.2 Functional Description

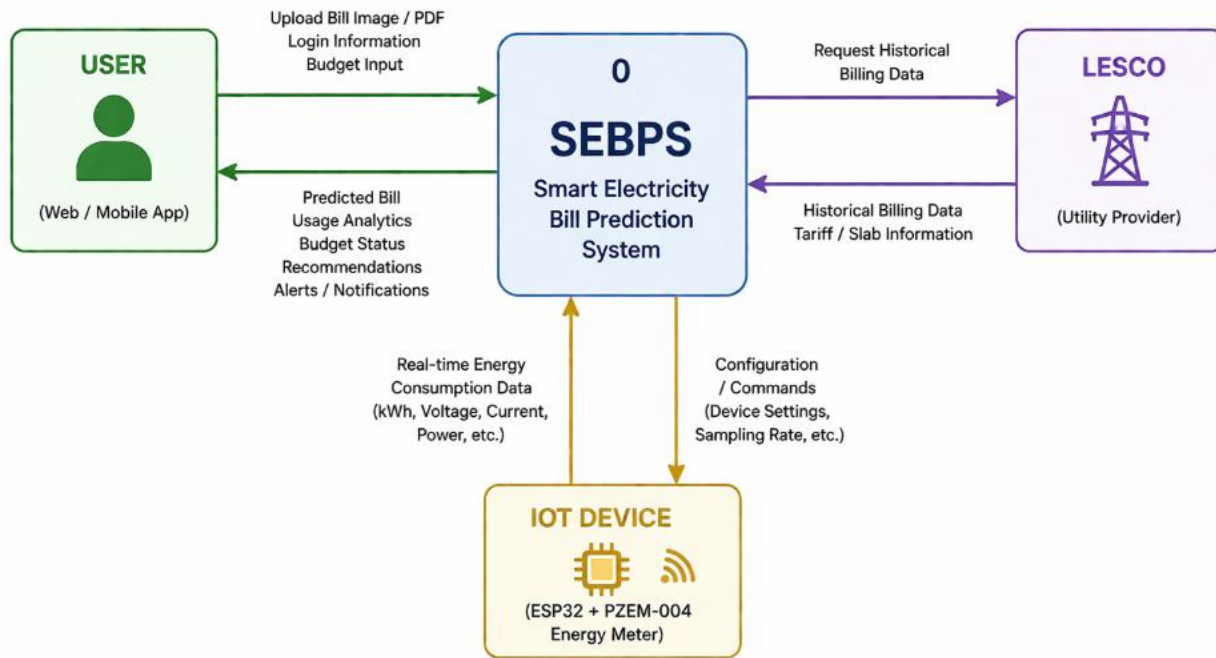
The current electricity billing process allows consumers to view their electricity consumption only after the monthly bill is generated. This makes it difficult for users to monitor daily electricity usage or estimate future bills in advance. Consumers often lack detailed information about which appliances contribute most to electricity costs, making effective energy management challenging.

The proposed Smart Electricity Bill Prediction System addresses these limitations by continuously collecting electricity consumption data from an ESP32 microcontroller connected to a PZEM-004T energy meter. Users can also manually configure household appliances, their power ratings, and operating hours. The system processes this information together with electricity tariff rates, taxes, fixed charges, and Fuel Price Adjustment (FPA) to estimate the monthly electricity bill.

The application also allows users to define monthly electricity budgets, monitor consumption trends, receive notifications when consumption exceeds predefined thresholds, and analyze historical electricity usage through graphical reports.

Context Diagram

SEBPS – CONTEXT DIAGRAM (DFD LEVEL 0)



2.3 User Classes and Characteristics

The Smart Electricity Bill Prediction System is intended for different categories of users based on their responsibilities and system access.

1. Residential Users (Primary Users)

Residential users are homeowners or tenants who wish to monitor electricity consumption and estimate monthly electricity bills. They have access to all application features including appliance management, budget planning, bill prediction, and real-time monitoring.

Characteristics:

- Limited technical knowledge
- Require simple and user-friendly interface
- Primary beneficiaries of the system
- Can manage multiple electricity meters

2. Administrators

Administrators manage the overall system configuration and maintenance. They monitor system performance, update tariff information, manage users if required, and ensure proper functioning of the application.

Characteristics:

- Technical knowledge of the system
- Responsible for maintenance and configuration
- Highest system privileges

3. Future Utility Providers (Optional Future Users)

Electricity distribution companies may utilize future versions of the system for customer engagement, electricity usage analysis, and demand forecasting.

Characteristics:

- Advanced technical users
- Access to aggregated electricity usage statistics
- Future enhancement of the proposed system

2.4 Design and Implementation Constraints

Several constraints influenced the design and implementation of the Smart Electricity Bill Prediction System.

1. Hardware Constraints

- The system relies on ESP32 and PZEM-004T hardware for real-time monitoring.
- Hardware accuracy depends on proper calibration.
- Stable Wi-Fi connectivity is required for communication.

2. Data Constraints

- LESCO provides only the previous twelve months of billing history.
- Tariff rates, taxes, and Fuel Price Adjustment (FPA) change periodically and require updates.

3. Software Constraints

- Mobile application compatibility with Android devices.
- Secure database storage for user information.
- Real-time synchronization between hardware and mobile application.

4. Security Constraints

- User authentication is required before accessing personal data.
- Secure communication between mobile application and cloud database.
- Protection of electricity consumption records and user information.

5. Development Constraints

- Limited project development timeline.

- Availability of IoT hardware components.
- Budget limitations for hardware procurement and testing.

2.5 Assumptions and Dependencies

The successful operation of the Smart Electricity Bill Prediction System depends on several assumptions and external dependencies.

Assumptions

- Users possess Android smartphones.
- Users have access to a stable internet connection.
- ESP32 and PZEM-004T devices remain properly connected and operational.
- Electricity tariff information is updated whenever changes occur.
- Users enter correct appliance information and operating hours for accurate bill estimation.
- Electricity supply remains available for continuous energy monitoring.

Dependencies

The system depends on the following external components:

- Firebase Authentication for user login.
- Firebase Firestore/Realtime Database for data storage.
- ESP32 microcontroller for IoT communication.
- PZEM-004T energy meter for real-time electricity measurement.
- LESCO tariff structure for electricity bill calculations.
- Internet connectivity for synchronization between mobile application and cloud services.
- OCR libraries (if enabled) for extracting data from electricity bills.

These assumptions and dependencies are essential for ensuring the accuracy, reliability, and continuous operation of the Smart Electricity Bill Prediction System. Future changes in hardware, tariff structures, or external services may require corresponding updates to the system.

Chapter 3. System Requirements

3.1 Functional Requirements

Use-Case Diagram

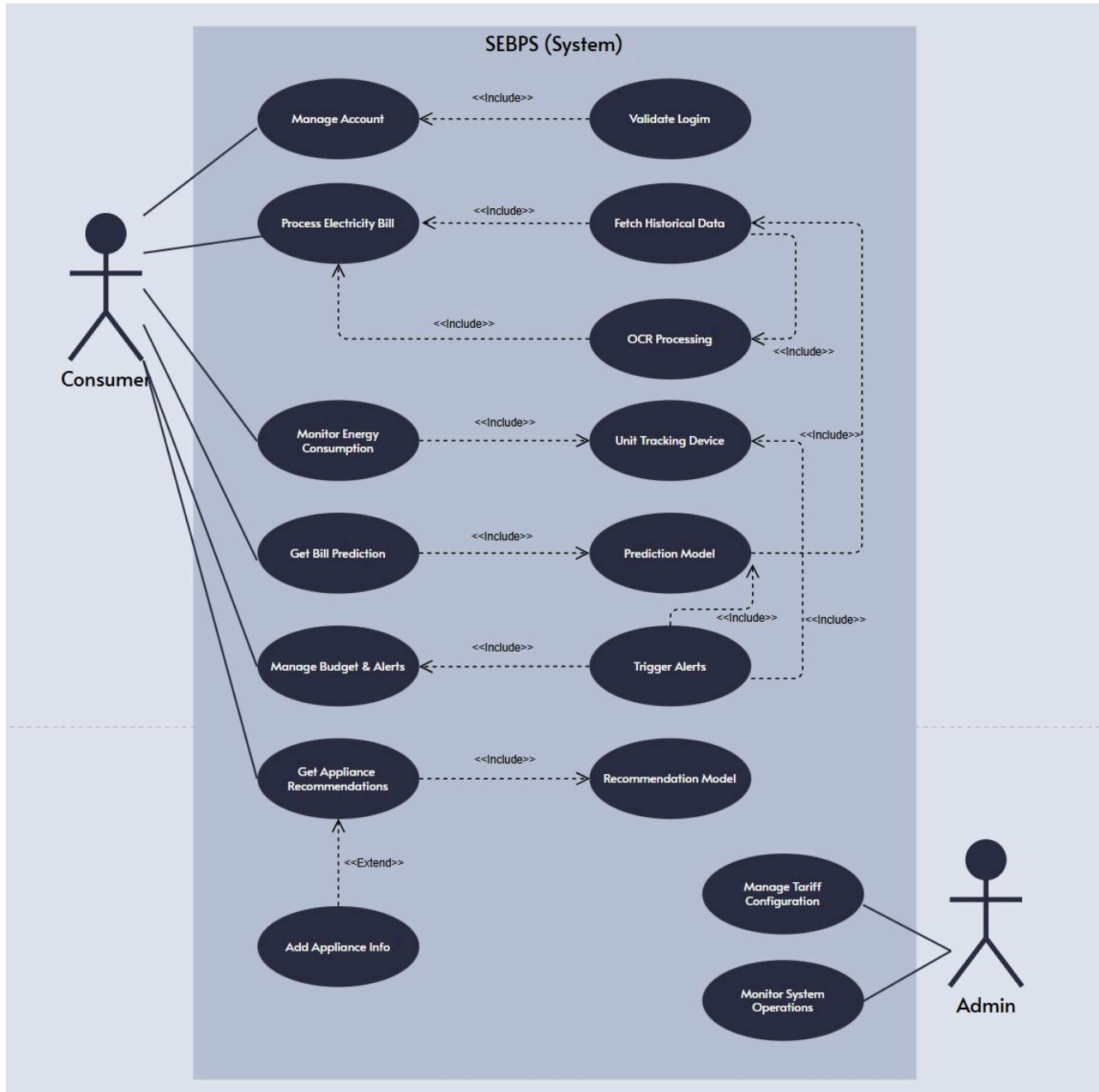


Figure 1: Use-Case Diagram

3.1.1 Upload and Extract Bill Data

Identifier	UC-1: Upload and Extract Bill Data	
Purpose	Allow users to upload electricity bill images and automatically extract consumer ID and billing information using OCR	
Priority	High	
Pre-conditions	User has a valid electricity bill image (LESCO format)	
Post-conditions	Consumer ID and relevant billing data extracted and stored in database	
Typical Course of Action		
S#	Actor Action	System Response
1	User uploads bill image via web/mobile interface	System validates image format and quality
2	User confirms upload	OCR module processes image and extracts consumer ID, billing date, units consumed
3	-	System displays extracted data for user verification
4	User verifies/corrects data	System saves data to database and links to user account
Alternate Course of Action		
S#	Actor Action	System Response
1	OCR fails to extract data	System prompts user to manually enter consumer ID
2	Image quality is poor	System suggests retaking photo with better lighting

Table 1: UC-1

3.1.2 Fetch Historical Billing Data

Identifier	UC-2: Fetch Historical Billing Data		
Purpose	Automatically retrieve one year of billing history from LESCO portal using extracted consumer ID		
Priority	High		
Pre-conditions	Consumer ID has been extracted/entered and is valid		
Post-conditions	12 months of historical billing data stored in database		
Typical Course of Action			
S#	Actor Action	System Response	
1	System receives consumer ID	Automated scraper	initiates

		connection to LESCO portal
2	-	System retrieves billing history for past 12 months
3	-	System parses and stores data (units, amount, tariff details)
4	User verifies/corrects data	System displays billing trends and consumption patterns
Alternate Course of Action		
S#	Actor Action	System Response
1	LESCO portal is down	System queues request and retries after 30 minutes
2	Consumer ID not found	System notifies user to verify ID and allows manual entry
3		
...		

Table 2: UC-2

3.1.3 Predict Future Electricity Bill

Identifier		UC-3: Predict Future Electricity Bill
Purpose		Generate accurate bill forecast using ML model based on historical data and current consumption patterns
Priority		High
Pre-conditions		Historical billing data available, ML model trained
Post-conditions		Predicted bill amount displayed with breakdown of charges
Typical Course of Action		
S#	Actor Action	System Response
1	User requests bill prediction	System retrieves historical consumption data
2	-	ML model analyzes patterns and predicts next month's units
3	-	System applies current tariff slabs, fuel adjustment, taxes
4	-	System displays predicted amount with detailed breakdown
5	User views forecast	System shows confidence interval and factors affecting prediction
Alternate Course of Action		
S#	Actor Action	System Response
1	Insufficient historical data	System uses regional average and notifies user of lower accuracy
2	User changes appliance settings	System recalculates prediction based

		on new inputs
3		
...		

Table 3: UC-3

3.1.4 Monitor Real-Time Consumption

Identifier		UC-4: Monitor Real-Time Consumption Future Electricity Bill	
Purpose		Track live electricity usage through IoT hardware (ESP32 + PZEM-004) and display on dashboard accurate bill forecast using ML model based on historical data and current consumption patterns	
Priority		High	
Pre-conditions		IoT hardware installed and connected to WiFi, linked to user account	
Post-conditions		Real-time consumption data displayed and logged in database	
Typical Course of Action			
S#	Actor Action		System Response
1	Appliances consume electricity		PZEM-004 measures voltage, current, power, energy
2	-		ESP32 processes data and transmits via WiFi every 5 seconds
3	-		Backend receives data and updates database
4	User opens dashboard		System displays live units consumed, current cost, and power usage
5			System updates graphs and trends in real-time
Alternate Course of Action			
S#	Actor Action		System Response
1	WiFi connection lost		ESP32 stores data locally and syncs when reconnected
2	Hardware malfunction detected		System alerts user and suggests troubleshooting steps
3			

Table 4: UC-4

3.1.5 Set Budget and Receive Alerts

Identifier	UC-5: Set Budget and Receive Alerts
-------------------	--

Purpose		Allow users to set monthly electricity budget and receive real-time alerts when approaching limit
Priority		Medium
Pre-conditions		User account active, real-time monitoring enabled
Post-conditions		Budget saved, alert system activated
Typical Course of Action		
S#	Actor Action	System Response
1	User sets monthly budget limit (e.g., PKR 5000)	System saves budget preference to user profile
2	-	System continuously monitors consumption vs budget
3	-	When 75% budget reached, system sends warning notification
4	-	When 90% reached, system sends urgent alert with reduction tips
5	User views budget status	System shows remaining budget, projected overage, days left
Alternate Course of Action		
S#	Actor Action	System Response
1	Budget exceeded	System suggests specific appliances to reduce usage
2	User disables alerts	System stops notifications but continues tracking
3		
...		

Table 5: UC-5

3.1.6 Get Appliance-Level Recommendations

Identifier	UC-6: Get Appliance-Level Recommendations		
Purpose	Provide personalized energy-saving recommendations based on appliance usage patterns		
Priority	Medium		
Pre-conditions	User has entered appliance details (wattage, usage hours)		
Post-conditions	Customized recommendations displayed to optimize energy consumption		
Typical Course of Action			
S#	Actor Action	System Response	
1	User adds appliances with wattage and daily usage	System calculates individual appliance cost contribution	

2	-	AI analyzes usage patterns and identifies inefficiencies
3	User views recommendations	System suggests: 'Reduce AC usage by 2hrs/day to save PKR 800'
4	-	System shows alternative usage schedules for cost optimization
5	User applies recommendation	System tracks savings and updates dashboard
Alternate Course of Action		
S#	Actor Action	System Response
1	No appliances added	System provides generic energy-saving tips
2	User rejects recommendations	System learns preference and adjusts future suggestions
3		
...		

Table 6: UC-6

3.1.7 Requirements Analysis and Modeling

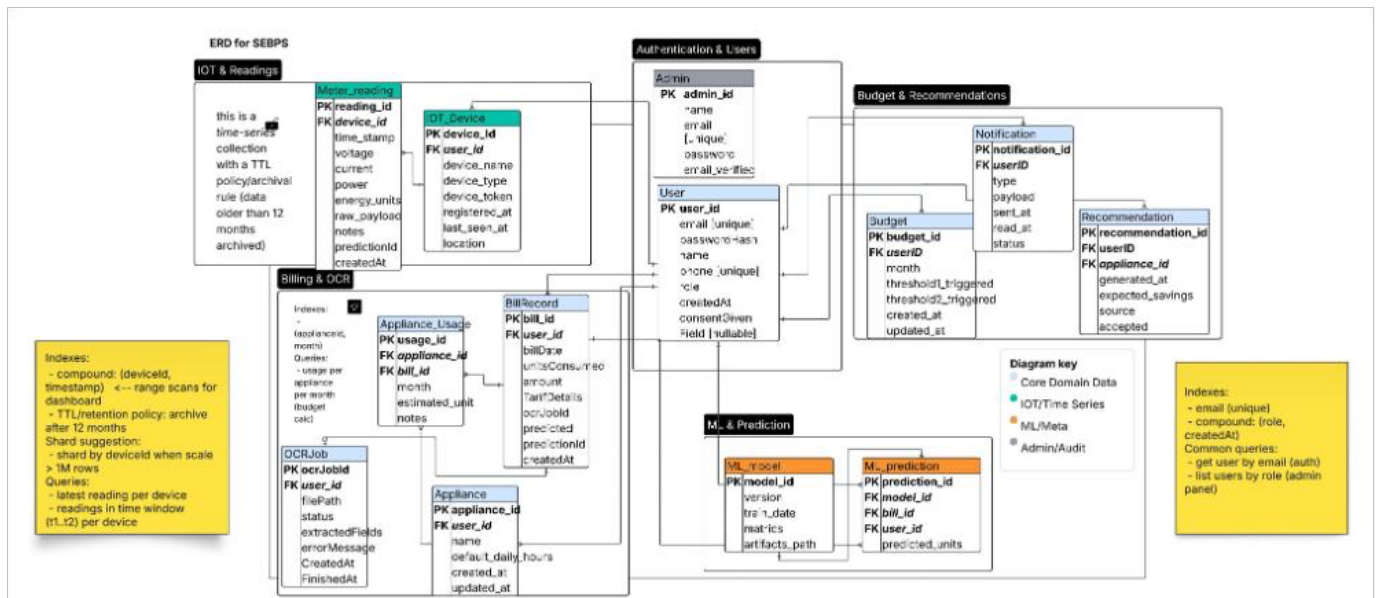


Figure 2: Entity-Relationship Diagram

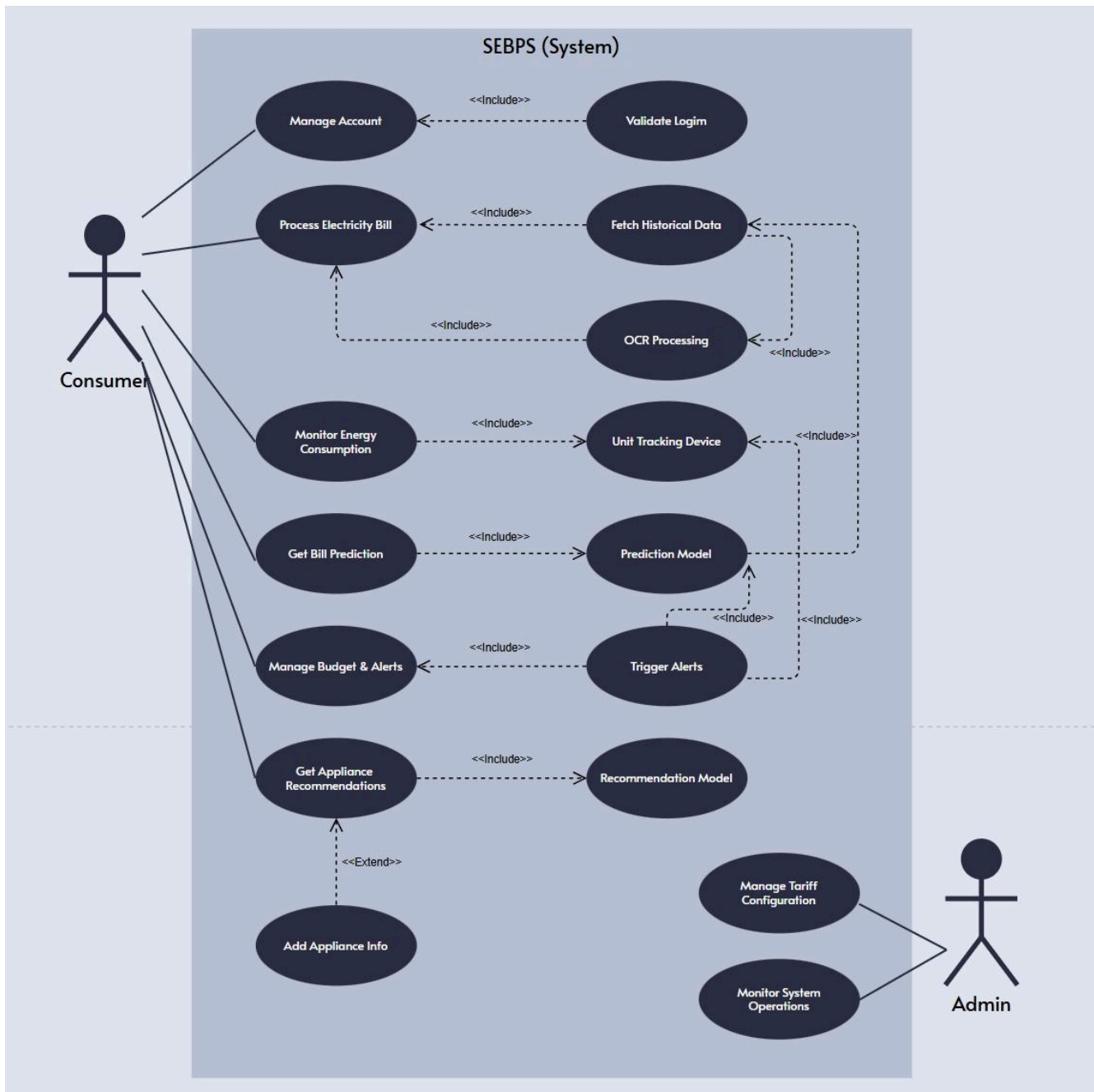


Figure 3: Use- Case Diagram



Smart Electricity Bill Prediction System (SEBPS)

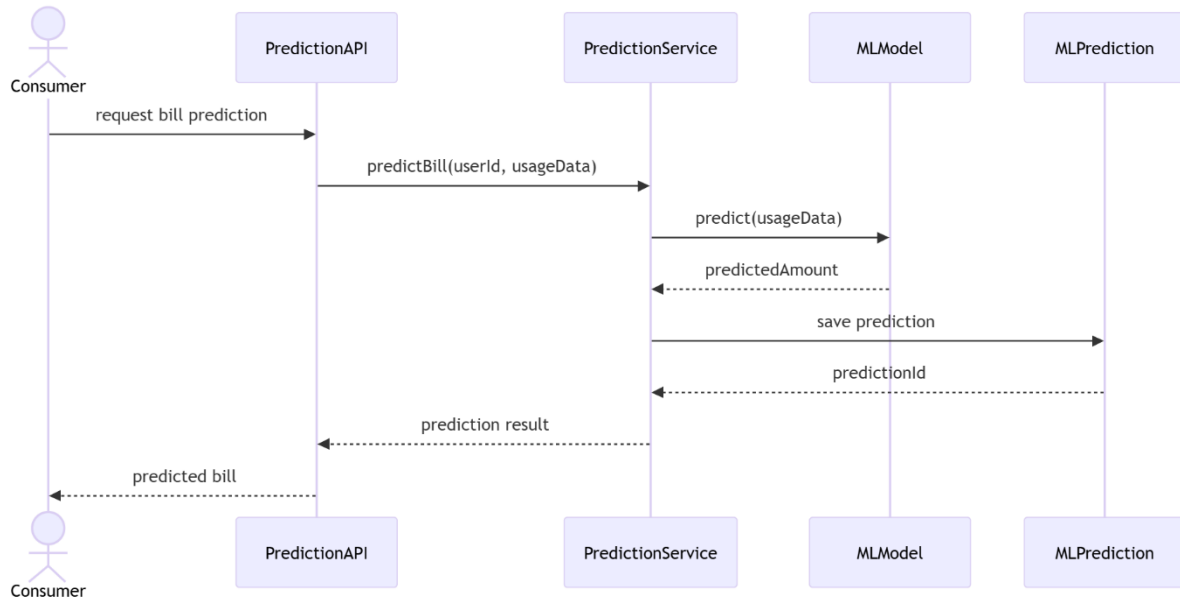


Figure 5: Sequence (Get Bill Prediction)

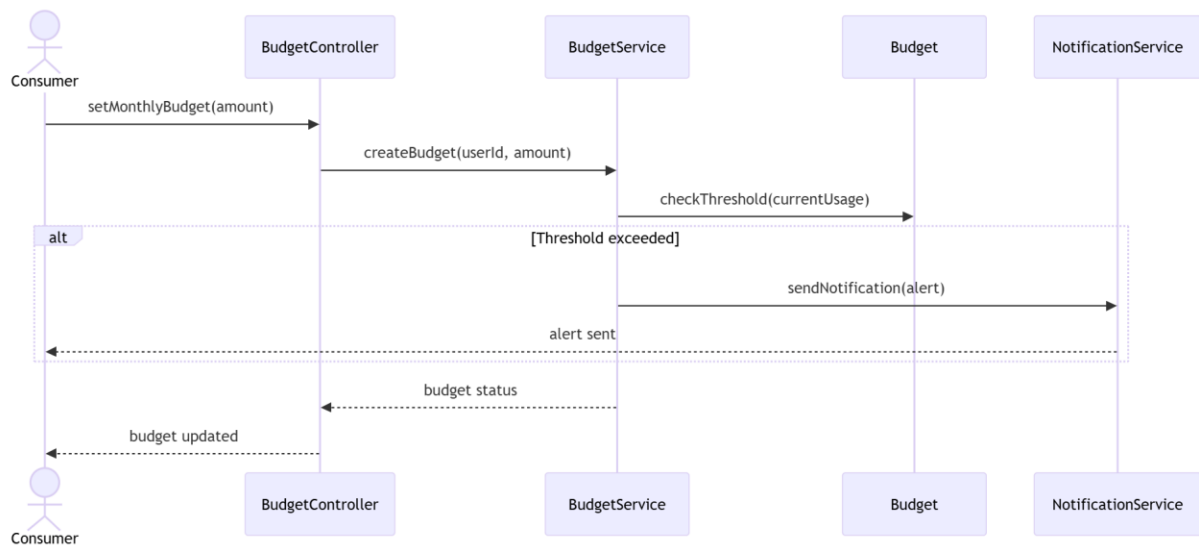


Figure 6: Sequence (Manage Budget Alerts)

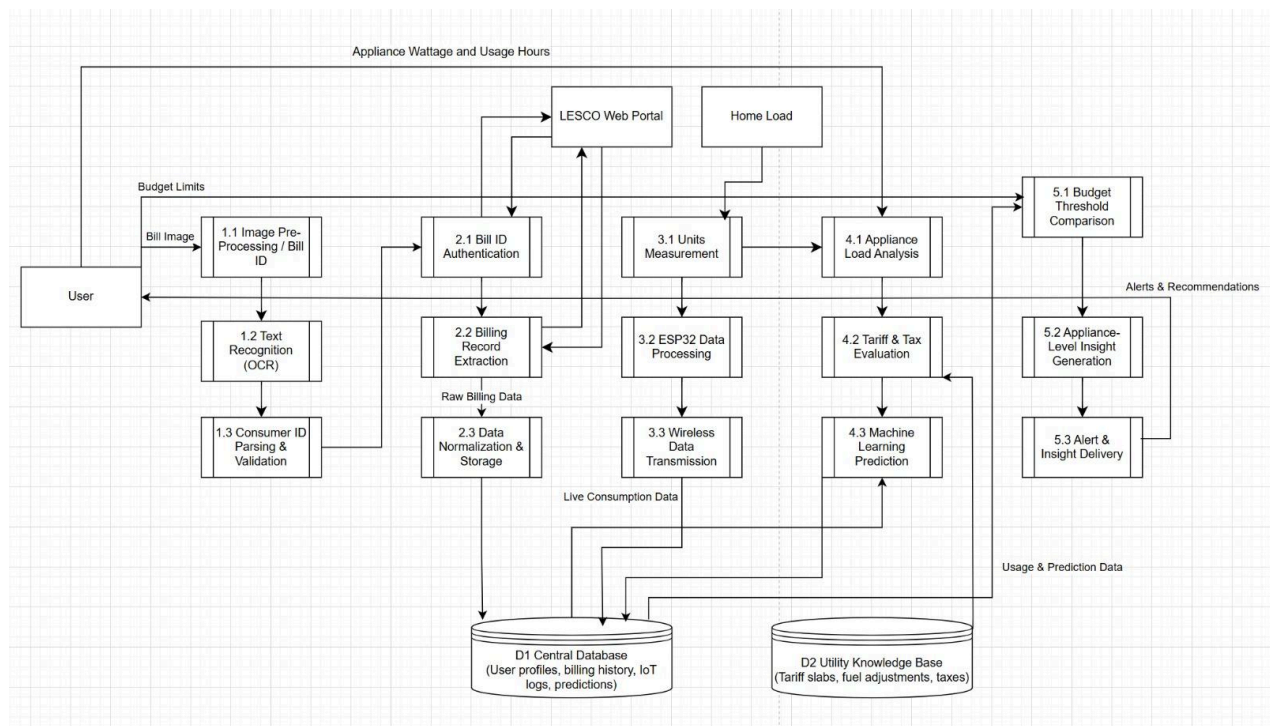
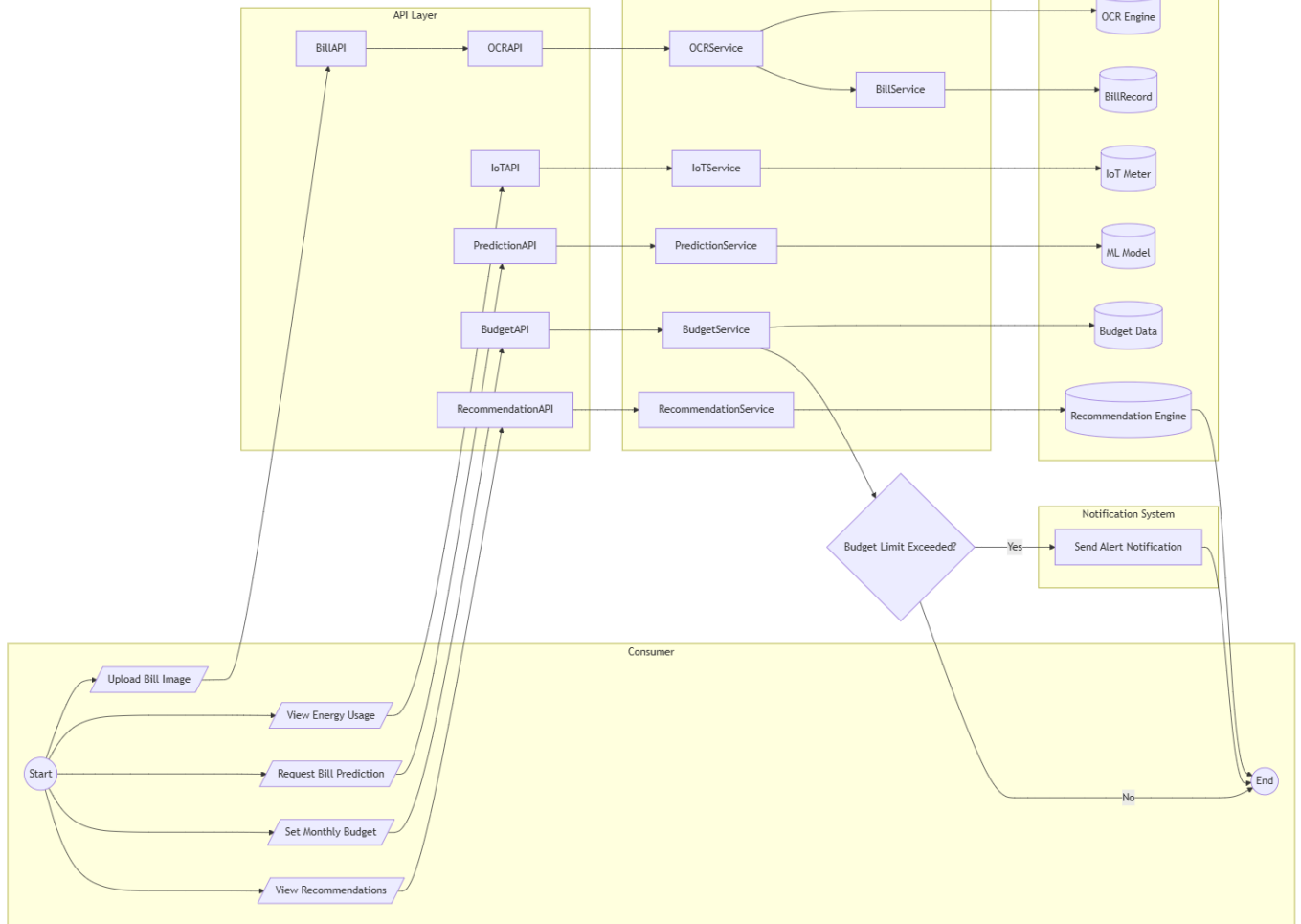


Figure 8: DFD (Data Flow Diagram)

3.2 Nonfunctional Requirements

3.2.1 Performance Requirements

The Smart Electricity Bill Prediction System (SEBPS) shall provide efficient and responsive performance to ensure a smooth user experience. The following performance requirements shall be satisfied:

- The system shall authenticate users within **3 seconds** under normal network conditions.
- The dashboard shall load user data and electricity consumption information within **3 seconds**.
- Real-time electricity consumption data received from the ESP32 and PZEM-004T energy meter shall be updated on the mobile application with minimal delay (typically within **2–5 seconds**, depending on network conditions).
- The system shall generate an estimated electricity bill within **5 seconds** after receiving all required inputs.
- Graphs and historical electricity consumption reports shall load quickly.
- The system shall support multiple registered users simultaneously without significant performance degradation.
- Database operations such as storing, retrieving, and updating user information shall be completed efficiently to ensure application responsiveness.
- The system shall maintain stable operation during continuous monitoring sessions without unexpected crashes or excessive battery consumption.

These performance requirements ensure that users receive timely electricity consumption updates and accurate bill predictions.

3.2.2 Safety Requirements

Although SEBPS is primarily a software application, it interacts with electrical monitoring hardware. Therefore, the following safety requirements shall be observed:

- The ESP32 and PZEM-004T energy meter shall be installed according to the manufacturer's safety guidelines.
- The application shall only read electricity consumption data and shall not directly control or switch electrical appliances.
- Users shall be informed whenever the hardware connection is lost or communication fails.
- Invalid or incomplete electricity readings shall not be used for bill prediction calculations.
- The application shall notify users when abnormal electricity consumption is detected.
- Hardware installation should be performed by qualified personnel to minimize electrical hazards.
- The application shall prevent users from entering invalid appliance power ratings or negative energy values.
- System failures shall not result in corruption or loss of stored electricity consumption records.

These safeguards help ensure reliable system operation while minimizing potential risks associated with hardware integration.

3.2.3 Security Requirements

The Smart Electricity Bill Prediction System shall protect user information and ensure secure communication between the web application, cloud database, and IoT devices.

The security requirements include:

- Every user shall be authenticated before accessing the system.
- User passwords shall be securely encrypted using Authentication mechanisms.
- Only authenticated users shall access their own electricity consumption records and personal information.
- Communication between the mobile application and cloud database shall use secure HTTPS connections.
- User data shall be stored securely within databases.
- The application shall prevent unauthorized modification of electricity consumption records.
- Session management shall automatically require users to log in again after logout.
- Personal information shall remain confidential and shall not be shared with unauthorized users.
- Database access permissions shall follow the principle of least privilege.
- Input validation shall be implemented to reduce the risk of invalid data entry and common security vulnerabilities.

These security measures ensure confidentiality, integrity, and availability of user data.

3.2.4 Additional Software Quality Attributes

The Smart Electricity Bill Prediction System has been designed to satisfy several important software quality attributes.

Usability

- The mobile application shall provide an intuitive and user-friendly interface.
- New users shall be able to understand the application's basic functionality with minimal guidance.
- Navigation between screens shall remain simple and consistent.

Reliability

- The application shall operate continuously with minimal failures during normal operation.
- Data shall be automatically synchronized whenever internet connectivity becomes available.
- The system shall recover gracefully from temporary communication failures.

Maintainability

- The software shall follow a modular architecture to simplify future maintenance.
- New features can be integrated without significant modification to existing modules.
- Source code shall follow consistent coding standards and documentation practices.

Scalability

- The system shall support future integration with additional electricity providers.
- Additional IoT devices and smart meters can be incorporated without redesigning the overall architecture.

Portability

- The mobile application shall support Android devices running Android 8.0 or later.

- Future deployment on other mobile platforms can be achieved with minimal architectural modifications.

Availability

- Cloud services shall remain accessible whenever internet connectivity is available.
- Offline data may be temporarily stored and synchronized when the connection is restored.

Flexibility

- Users shall be able to modify appliance information, electricity tariff values (where applicable), and monthly budgets without affecting other system functions.

Robustness

- The application shall handle invalid user input without crashing.
- Unexpected hardware disconnections shall not terminate the application.

Reusability

- Software modules such as authentication, user management, and dashboard components shall be reusable for future projects and system enhancements.

Chapter 4. Technical Architecture

The Smart Electricity Bill Prediction System (SEBPS) is a custom-built, cloud-enabled application that integrates Internet of Things (IoT), Optical Character Recognition (OCR), Artificial Intelligence (AI), and modern web and mobile technologies to provide real-time electricity monitoring, bill prediction, appliance management, and energy budgeting. The architecture follows a layered client-server model that separates presentation, business logic, intelligent processing, and persistent storage into independent modules. This modular approach improves maintainability, scalability, and future extensibility.

The system performs both online transaction processing and analytical processing. Online operations include user authentication, IoT data acquisition, appliance management, OCR-based bill uploads, and dashboard updates, while analytical operations include electricity bill prediction, budget analysis, energy consumption forecasting, and recommendation generation.

The architecture consists of the following major subsystems:

- Web Dashboard
- Mobile Application
- REST API Layer
- Authentication Module
- OCR Processing Module
- Machine Learning Prediction Engine
- Billing Engine
- Budget Management Module
- IoT Monitoring Module
- Notification Service
- Recommendation Engine
- AI Chatbot
- Cloud Database
- ESP32–PZEM-004 Energy Meter

All services communicate through secure REST APIs and cloud services. The backend is hosted on cloud infrastructure while IoT devices communicate over Wi-Fi using secure HTTPS requests. User information, billing records, IoT readings, prediction history, appliance configurations, and notification logs are securely maintained in the cloud database.

Smart Electricity Bill Prediction System (SEBPS)

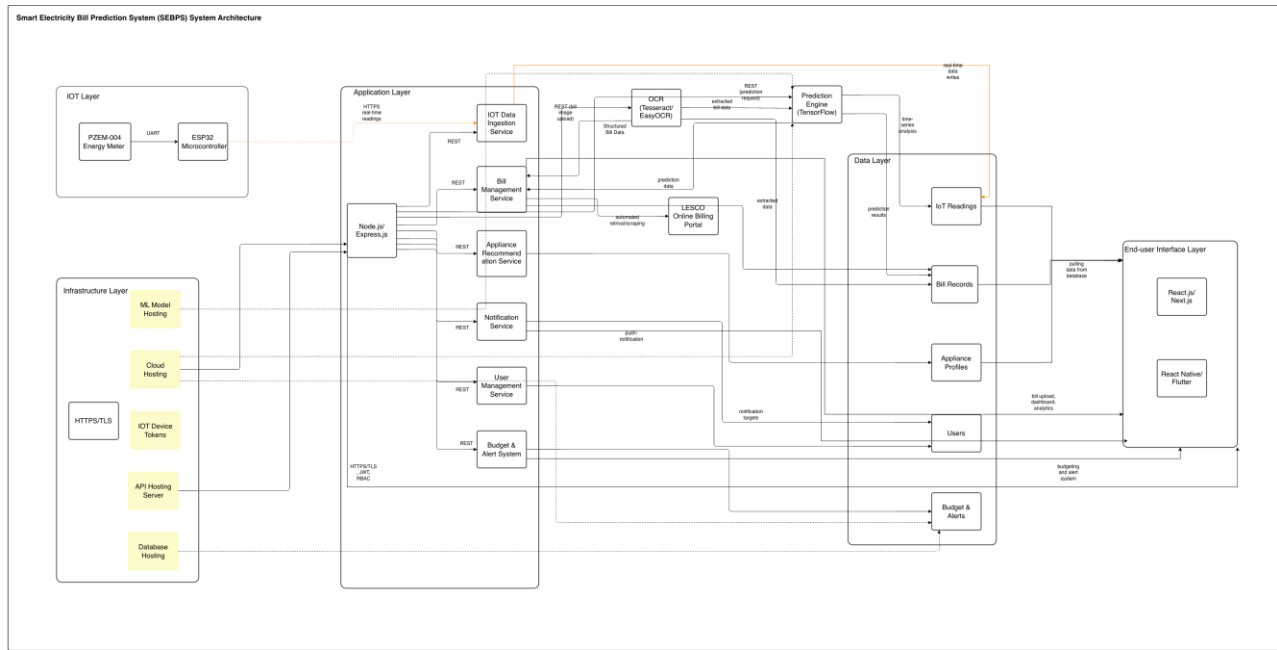


Figure 9: Architecture Diagram

4.1 Application and Data Architecture

SEBPS follows a layered, client server architecture split into five tiers: the End User Interface Layer, the Application (API) Layer, the Service Layer, the Model and Device Layer, and the Data Layer. Each tier has its own clear job and talks to the layers next to it through defined contracts, which keeps the parts loosely coupled and easier to test on their own.

4.1.1 Layered Architecture Overview

- **End User Interface Layer:** The React.js web dashboard and the mobile view. Users interact with bill predictions, IoT readings, the appliance list, and budget alerts through these screens.
- **Application Layer (API Gateway):** A Node.js and Express.js REST API server that takes in all client requests, validates the data, checks the user's JWT token, and routes the request to the right service.
- **Service Layer:** The business logic, split into the OCR Service, Billing Service, Prediction Service, IoT Service, Budget Service, Recommendation Service, Notification Service, and Chatbot Service. Each one is kept separate and reaches the data layer on its own.
- **Model and Device Layer:** The machine learning prediction engine, the OCR engine (EasyOCR with OpenCV pre-processing), the IoT meter firmware (ESP32 and PZEM-004), and the recommendation logic.
- **Data Layer:** The cloud database holding user profiles, billing records, IoT readings, appliance data, model metadata, budget settings, and notification logs.

4.1.2 Data Flow Summary

Data enters the system through two paths. The first is started by the user: a consumer uploads a bill image, the OCR service pulls out the consumer ID, and the billing service retrieves the past months of records. The second is started by the hardware: the ESP32 keeps sampling live readings from the PZEM-004 meter and sends them over Wi-Fi to the IoT service, which checks them and stores them. Both paths feed into the prediction engine, which takes the historical bills, the live IoT readings, and the appliance load profiles and produces a forecast for the month. The forecast and the budget check are then sent back up to the dashboards through secure API calls. The notification service watches the budget thresholds and pushes an alert when consumption gets close to or crosses the limit.

4.1.3 Database Schema Design

The database is organized into logical groups that match the system's features. The Users group stores account details, roles, and consent flags. The IoT Readings group keeps a time ordered set of device readings, indexed on device ID and timestamp so the dashboard can query them quickly, with older readings archived after a set period. The Billing and OCR group holds OCR job records, the extracted bill data, and appliance usage logs. The Prediction group stores model metadata and the individual prediction results linked back to the bill they came from. The Budget and Recommendations group keeps the monthly budgets, the threshold flags, the recommendation items, and the expected savings. Field names use snake_case and timestamps follow the ISO 8601 format so everything stays consistent.

4.2 Component Interactions and Collaborations

This part describes how the major components actually work together at runtime. Four main flows are covered: bill processing through OCR, real time monitoring through IoT, bill prediction through machine learning, and budget alerts.

4.2.1 OCR Based Bill Processing Flow

When a consumer uploads a bill image, the BillController takes the file and hands it to the OCRController. The OCRController calls the OCRService, which cleans up the image first (grayscale, noise reduction, and adaptive thresholding through OpenCV) before passing it to the EasyOCR engine. The extracted text is parsed to find the consumer reference number. That reference goes to the BillingService, which retrieves the past billing records, stores them as structured records in the database, and returns a confirmation with a short summary of the history that was pulled.

4.2.2 IoT Real Time Monitoring Flow

The ESP32 samples voltage, current, power, and accumulated energy from the PZEM-004 over a serial link at a set interval. Each reading is turned into a small JSON payload and sent to the backend over HTTPS, with the device authenticated by a pre-shared token. The IoTController checks the token and passes the reading to the IoTService, which works out the consumption in kilowatt hours and

stores it. The dashboard listens for live updates through a socket connection, so the graphs update as new readings come in without the page having to keep asking the server.

4.2.3 Machine Learning Bill Prediction Flow

When a consumer asks for a prediction, the request reaches the PredictionService with the user ID and the latest usage data. The service gathers the consumer's past bills, the recent IoT readings, and the appliance load profiles, builds a feature set, and runs the trained model for that user. The predicted units are then passed through the slab based tariff calculator, which applies the current slab rates, fuel adjustment charge, and taxes step by step to reach a final estimated amount. The result is saved and returned to the user, and if the estimate goes over the budget the recommendation service is triggered.

4.2.4 Budget Alert Management Flow

When a consumer sets or updates a monthly budget, the BudgetController stores the budget and sets up trigger flags at 75 percent and 100 percent of the amount. The BudgetService keeps an eye on the running consumption by listening to the IoT data. When the projected cost crosses a threshold, the matching flag is set and the NotificationService sends an alert to the user and marks it as sent. The dashboard then updates the budget progress widget to match.

4.2.5 AI Chatbot Flow

When a user sends a message to the chatbot, the request reaches the ChatbotController with the user ID and the question. The ChatbotService pulls together the context for that user, which includes the latest IoT readings, the most recent prediction, and a short summary of the recent bills, and builds a prompt that pairs this context with an instruction that keeps the assistant focused on electricity and billing topics. The prompt is sent to a large language model through an API, and the reply is checked and returned to the user. Because the live unit data is part of the context, the chatbot can answer questions like how many units have been used so far today, why the bill might be higher this month, or which appliance is driving the cost, instead of giving a generic response. If a question falls outside energy and billing, the assistant steers the user back to what the system can actually help with.

4.2.6 API Endpoint Catalogue

Method	Endpoint	Module	Description
POST	/api/auth/register	Auth	Register new consumer account
POST	/api/auth/login	Auth	Authenticate user, return JWT token
POST	/api/ocr/upload	OCR	Upload bill image, extract consumer ID

Method	Endpoint	Module	Description
GET	/api/bills/history/:userId	Billing	Retrieve historical billing records
POST	/api/predict/bill	ML	Request AI-based bill prediction
GET	/api/iot/latest/:deviceId	IoT	Fetch latest meter reading from ESP32
POST	/api/appliances/add	Appliance	Add household appliance with wattage
GET	/api/budget/:userId	Budget	Get budget status and threshold alerts
PUT	/api/budget/update	Budget	Update monthly budget limit
GET	/api/recommendations/:userId	ML	Get energy-saving recommendations

Table 7: Primary REST API Endpoints

4.3 Design Reuse and Design Patterns

SEBPS was built with reuse and maintainability in mind from the start. Standard design patterns were used across the layers to cut down on repeated code and keep the system open to extension later.

4.3.1 Applied Design Patterns

The following design patterns are formally applied within SEBPS:

Pattern	Category	Applied In	Benefit
MVC / MVVM	Structural	Web & Mobile Frontend	Separation of UI, logic, and data layers
Repository Pattern	Data Access	Backend Services	Abstracts database operations cleanly
Observer / Event-Driven	Behavioral	IoT Data Pipeline	Reactive updates from ESP32 readings
Singleton	Creational	Database Connection	Single shared DB instance across modules
Factory Method	Creational	ML Model Loader	Dynamic instantiation

Pattern	Category	Applied In	Benefit
			of prediction models
Strategy Pattern	Behavioral	Billing Logic Engine	Interchangeable tariff slab algorithms
Facade	Structural	API Gateway Layer	Simplified interface for complex subsystems
Decorator	Structural	Middleware / Auth	Wraps request handlers with auth & logging

Table 8: Data Reused

4.3.2 Reused Libraries and Components

To accelerate development and reduce risk, SEBPS reuses a curated set of open-source libraries and third-party components in place of custom implementations. The table below catalogues the primary reused components, their origins, and their specific roles within the system.

Component / Library	Source	Usage in SEBPS
EasyOCR	Open Source (Python)	Extracts consumer ID and billing data from bill images
Recharts / Chart.js	Open Source (JS)	Renders consumption graphs and prediction charts
Firebase SDK	Google LLC	Authentication, real-time DB, and cloud hosting
scikit-learn	Open Source (Python)	Training and inference of ML prediction models
Mongoose / Prisma ORM	Open Source (JS)	Database schema modeling and query abstraction
Axios / Fetch API	Open Source (JS)	HTTP communication between frontend and backend
Socket.IO	Open Source (JS)	Real-time push updates for IoT meter readings
ESP32 Arduino Libraries	Espressif Systems	Wi-Fi, UART communication, PZEM data parsing
React Native Paper	Open Source	Pre-built mobile UI components and theming
JWT (jsonwebtoken)	Open Source (JS)	Secure token-based user authentication

Table 9: Libraries Used

4.4 Technology Architecture

The technology architecture of SEBPS defines the specific technologies, frameworks, protocols, and infrastructure components selected for each tier of the system. Technology choices were governed by three criteria: compatibility with the academic timeline, availability as open-source or free-tier tools, and fitness for the technical requirements of each module.

4.4.1 Full Technology Stack

Layer	Component	Technology	Responsibility
Frontend (Web)	React.js / Next.js	React.js + TailwindCSS	UI dashboards, charts, forms
Frontend (Mobile)	React Native + Flutter	Flutter (Dart)	Cross-platform mobile app
Backend	Node.js / Express	Node.js + Express.js	REST APIs, business logic
ML Module	Python ML Engine	TensorFlow / scikit-learn	Bill prediction model
OCR Module	Tesseract / EasyOCR	EasyOCR + OpenCV	Bill image parsing
Database	Cloud Database	Firebase / MongoDB	Persistent data storage
IoT Layer	ESP32 + PZEM-004	C++ (Arduino IDE)	Real-time energy metering
Hosting	Cloud Platform	Firebase / Render	Scalable cloud deployment
Auth	Authentication	Firebase Auth / JWT	User session management

Table 10: SEBPS Full Technology Stack

4.4.2 Frontend Architecture

The web dashboard is developed using React.js with Next.js for server-side rendering, providing fast initial page loads and SEO-friendly routing. Tailwind CSS is used for utility-first styling to maintain visual consistency across all pages. Recharts is integrated for rendering interactive consumption and prediction graphs. The mobile application is developed using Flutter, enabling a single codebase to target both Android and iOS platforms. State management on both platforms follows the Provider pattern, where each service (billing, IoT, prediction, budget) exposes a ChangeNotifier that drives reactive UI updates.

4.4.3 Backend Architecture

The backend is implemented as a monolithic Node.js application using the Express.js framework, structured internally as a modular monolith to support future decomposition into microservices if scale demands it. Each functional domain (OCR, billing, prediction, IoT, budget, recommendations) is implemented as an independent Express Router with its own controller, service, and data access layers. Cross-cutting concerns such as authentication, request logging, error handling, and input validation are implemented as Express middleware applied globally. The Python-based ML module runs as a separate process exposed via a lightweight Flask REST API, invoked by the Node.js backend through an internal HTTP call, isolating the machine learning environment from the JavaScript runtime.

4.4.4 IoT Communication Architecture

The ESP32 microcontroller communicates with the PZEM-004 energy meter over UART (Universal Asynchronous Receiver-Transmitter) using the ModBus protocol at a baud rate of 9600. Sampled readings are transmitted to the backend server over Wi-Fi using HTTPS POST requests secured with TLS 1.2, with the device authenticated via a pre-shared bearer token stored in the ESP32's non-volatile storage. On the server side, incoming IoT data is processed asynchronously to avoid blocking the main event loop. Socket.IO is used to broadcast new readings to connected web and mobile clients in real time, providing a seamless live monitoring experience without consumer polling.

4.4.5 Deployment Architecture

The backend API and database are hosted on Firebase Cloud Functions and Firebase Firestore respectively, providing automatic scaling and a generous free-tier allocation suitable for the project's academic scope. The ML Flask microservice is deployed separately on Render or a Python-compatible cloud instance. The frontend web application is deployed via Vercel, which offers continuous deployment from the Git repository and a global CDN for fast content delivery. Mobile application distribution is managed through direct APK installation for testing and Google Play internal testing for structured user evaluation. All inter-service communication is encrypted using HTTPS/TLS. Environment variables and API keys are managed through platform-specific secret management services and are never committed to version control.

4.5 Architecture Evaluation

The SEBPS architecture is evaluated against eight quality attributes derived from the system requirements defined in Phase I and Phase II. The evaluation methodology is inspired by the Architecture Tradeoff Analysis Method (ATAM), where each quality attribute is assessed through a concrete scenario, the corresponding architectural decision is identified, the expected outcome is stated, and the residual risk level is rated.

Quality Attribute	Scenario	Architecture Decision	Expected Outcome	Risk Level
Performance	IoT data from 100+ concurrent devices	Async event-driven backend via Node.js + Socket.IO	Sub-second latency for live readings	Low
Scalability	User base grows from 500 to 50,000	Microservice-ready modular backend on cloud hosting	Horizontal scaling with minimal refactoring	Low
Reliability	Backend server crashes during peak hours	Cloud hosting with auto-restart & health monitoring	System recovery within 30 seconds	Medium
Security	Unauthorized access to user billing data	JWT auth, HTTPS/TLS, Firebase security rules	Zero unauthorized data exposure	Low
Accuracy	ML model drift over seasonal changes	Periodic model retraining with new historical data	Prediction error maintained below 10%	Medium
Maintainability	Tariff slabs updated by NEPRA	Strategy pattern isolates billing logic module	Update deployable without full system restart	Low
Portability	Expand beyond LESCO to other DISCOs	Configurable billing engine, DISCO-agnostic DB schema	New DISCO onboarding in under 2 weeks	Medium
Usability	Non-technical user struggles with OCR upload	Intuitive onboarding flow with step-by-step guidance	Task completion rate above 90% in user testing	Low

Table 11: Architecture Evaluation Against Quality Attributes

4.5.1 Critical Tradeoff Analysis

The most significant architectural tradeoff in SEBPS is the decision to implement the backend as a modular monolith rather than a fully distributed microservices architecture. A microservices approach

would offer superior independent scalability per service, but it introduces substantial operational complexity, including service discovery, distributed tracing, inter-service authentication, and network latency and that is disproportionate to the academic project's timeline and team size. The modular monolith preserves clear domain boundaries through internal module separation while remaining deployable as a single unit, providing a pragmatic balance between architectural cleanliness and delivery velocity.

A secondary tradeoff arises from the choice to run the ML inference engine as a separate Python Flask process rather than integrating a JavaScript-native model. While this introduces a cross-process HTTP hop, it enables the team to leverage the full Python scientific computing ecosystem (TensorFlow, scikit-learn, pandas, NumPy) without constraint, which is essential for achieving acceptable prediction accuracy. The latency overhead introduced by this inter-process call is estimated at under 200 milliseconds for a single prediction request, which is acceptable for a non-real-time operation.

4.5.2 Known Architectural Risks

The following risks have been identified and are being actively mitigated:

- **LESCO Portal Dependency:** Automated historical data retrieval via web scraping is vulnerable to portal layout changes or captcha enforcement. Mitigation: The system is designed to fall back gracefully to manual consumer ID entry, and the scraping module is encapsulated to allow rapid replacement.
- **IoT Network Instability:** The ESP32 relies on stable Wi-Fi connectivity. Packet loss or disconnections may introduce gaps in the time-series data. Mitigation: The firmware implements an exponential back-off reconnection strategy and a local circular buffer that stores up to sixty readings pending transmission.
- **ML Model Accuracy Degradation:** Prediction accuracy depends on the quality and volume of historical billing data. New users with fewer than three months of history will receive lower-confidence predictions. Mitigation: The system displays a confidence indicator alongside each prediction and falls back to a rule-based tariff slab calculator for users with insufficient history.
- **Data Privacy:** Consumer billing data is sensitive. Mitigation: All data at rest is encrypted via Firebase's built-in AES-256 encryption, and data in transit is protected by TLS 1.2. Users may delete their account and all associated data through the profile settings.

4.5.3 Architecture Fitness for Purpose

Overall, the SEBPS architecture is assessed as fit for purpose. It satisfies the primary functional requirements of bill prediction, real-time energy monitoring, OCR-based data entry, budget management, and appliance-level recommendations within the constraints of the academic project's

scope, timeline, and available resources. The modular structure ensures that individual components such as the billing engine, ML model, and IoT module can be developed, tested, and debugged in isolation, which is aligned with the team's parallel development workflow. The chosen cloud-hosted deployment model provides the availability and accessibility required for user evaluation without requiring dedicated on-premises infrastructure.

Chapter 5. Detailed Design and Implementation

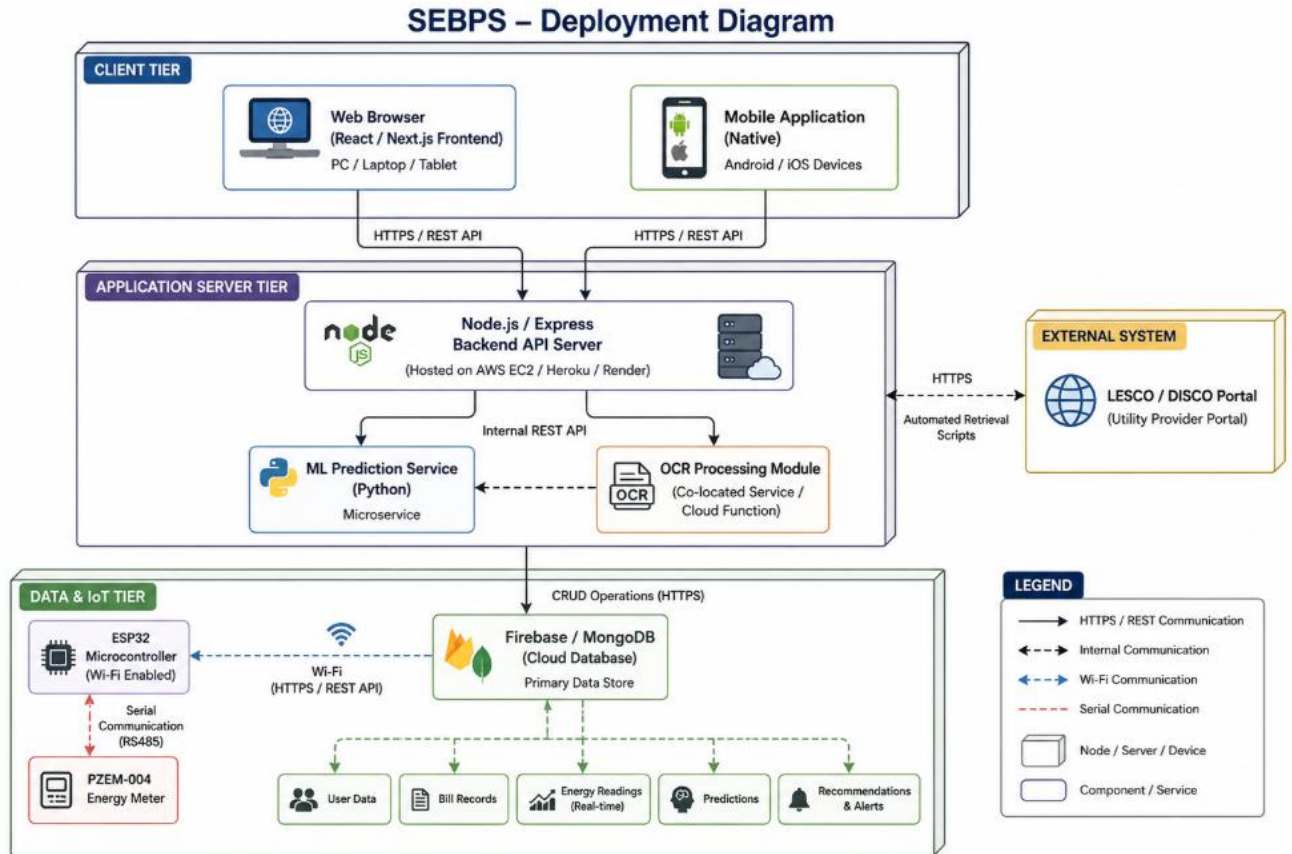
SEBPS uses a modular, layered design where independent components talk to each other through clear interfaces. This kept the system easy to maintain and made it simple to plug in the IoT and machine learning parts without disturbing the rest.

5.1 Component-Component Interface

The frontend never reaches the database directly. It only ever calls the backend REST API, which then talks to the services and the database. The IoT firmware posts readings to a dedicated endpoint, and the Node backend calls the Python model service over an internal HTTP call when a prediction is needed. Keeping these interfaces narrow meant a change inside one component rarely forced a change in another.

5.2 Component-External Entities Interface

The main external touch points are the LESCO bill portal, from which historical records are retrieved, and the ESP32 device, which is treated as an external client that authenticates with a device token. SEBPS does not connect to any bank or payment system, since billing payment is out of scope.

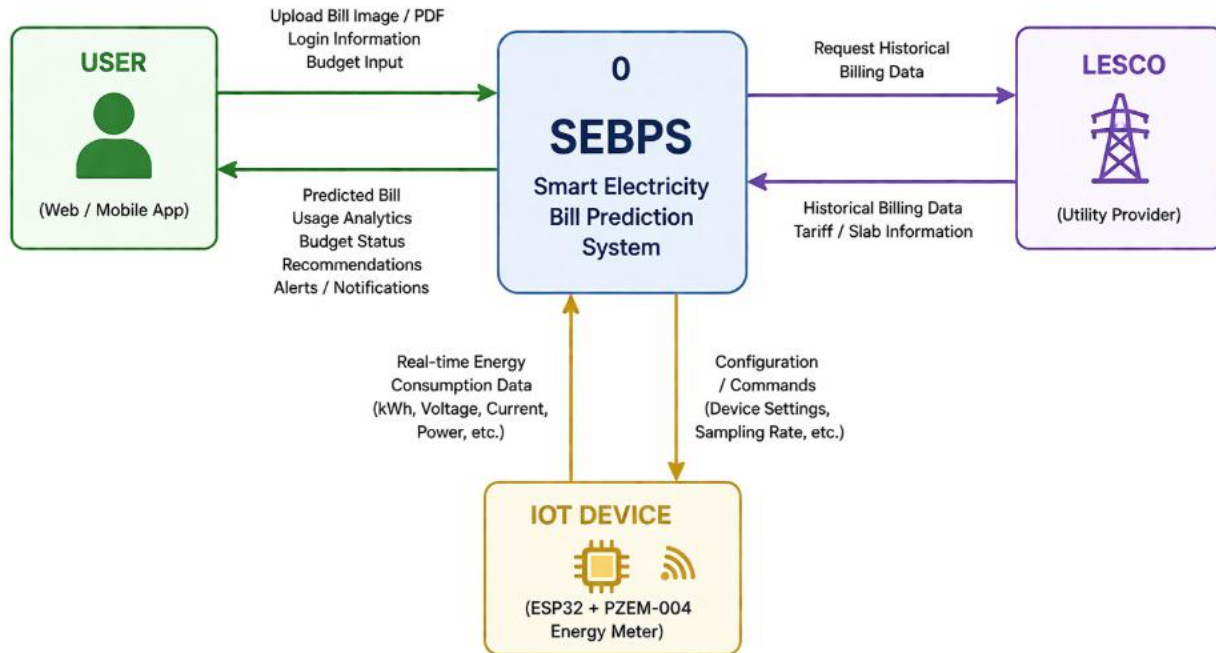


External Entities

- Electricity Provider System (LESCO)
- IoT Hardware System
- Cloud Database Services

Context Diagram

SEBPS – CONTEXT DIAGRAM (DFD LEVEL 0)

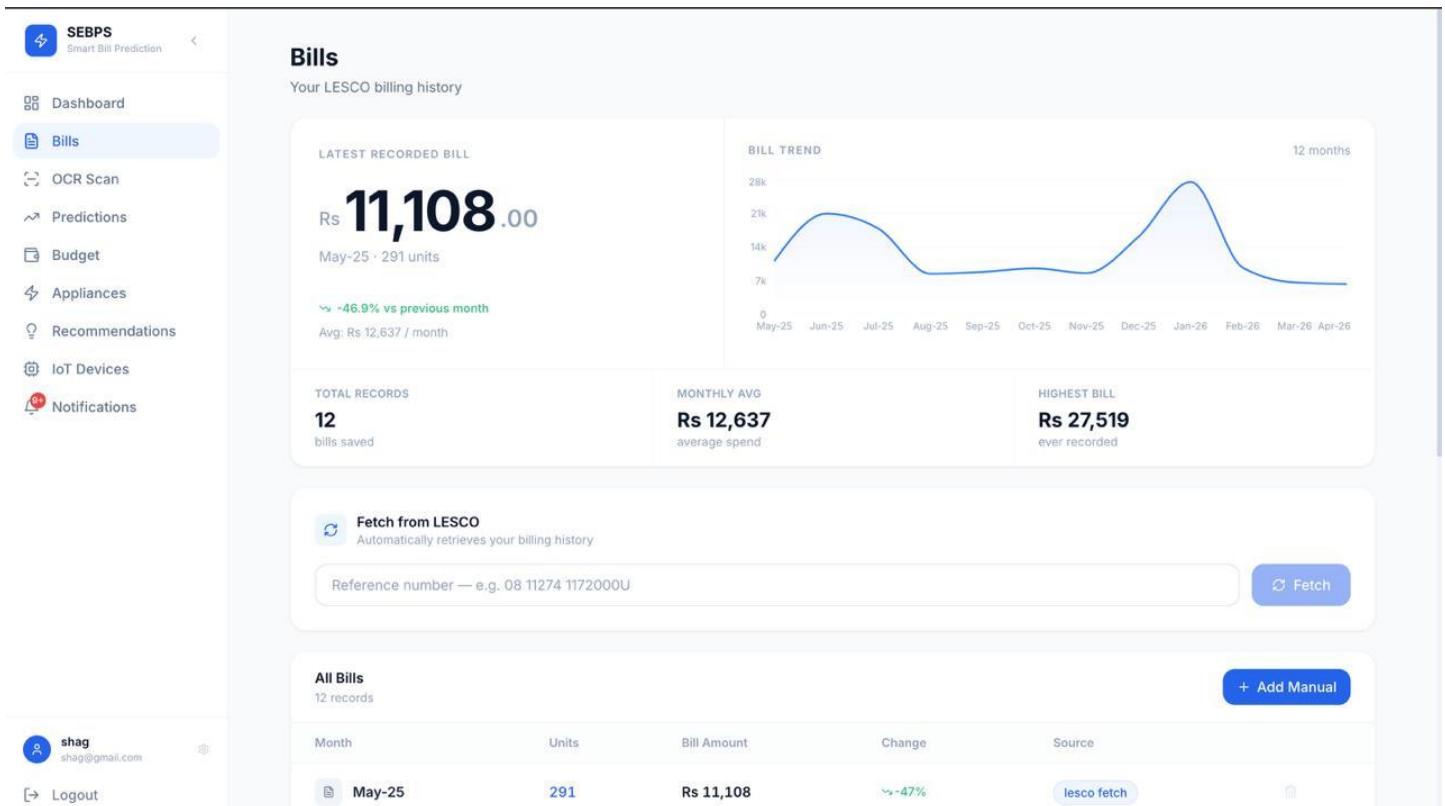


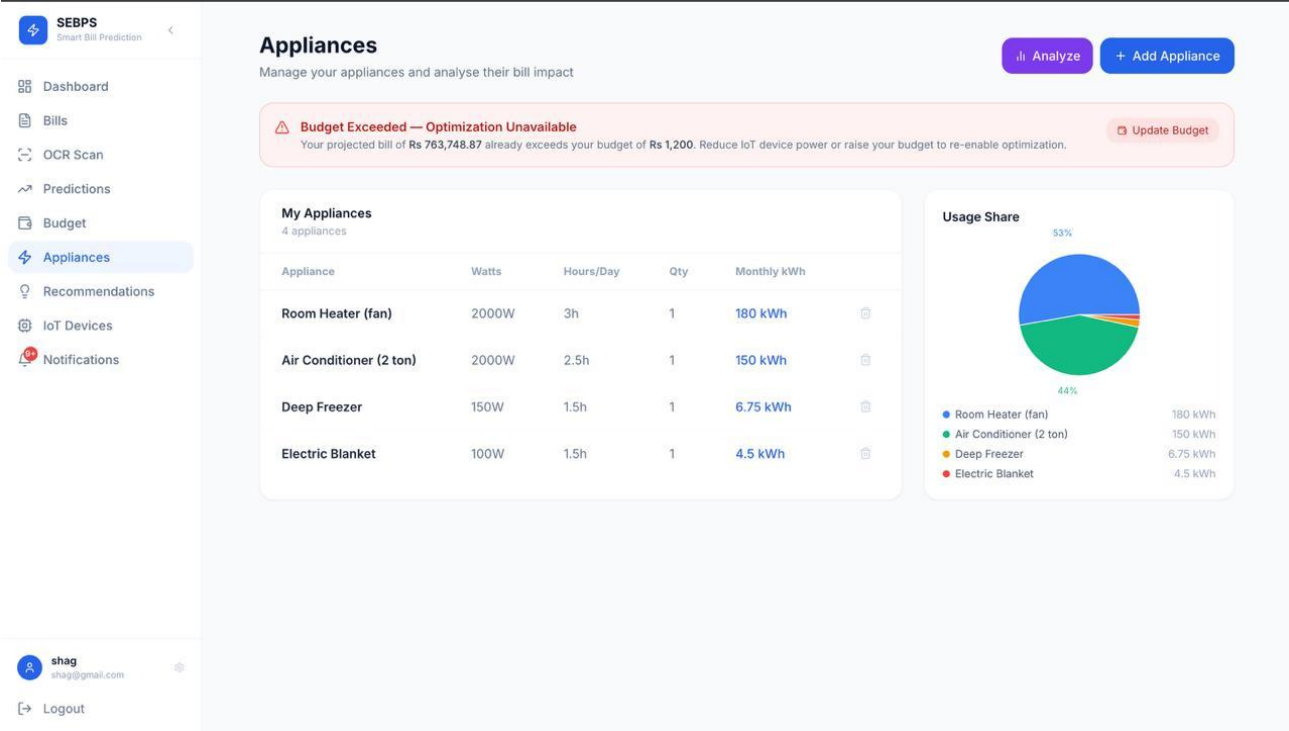
5.3 Component-Human Interface

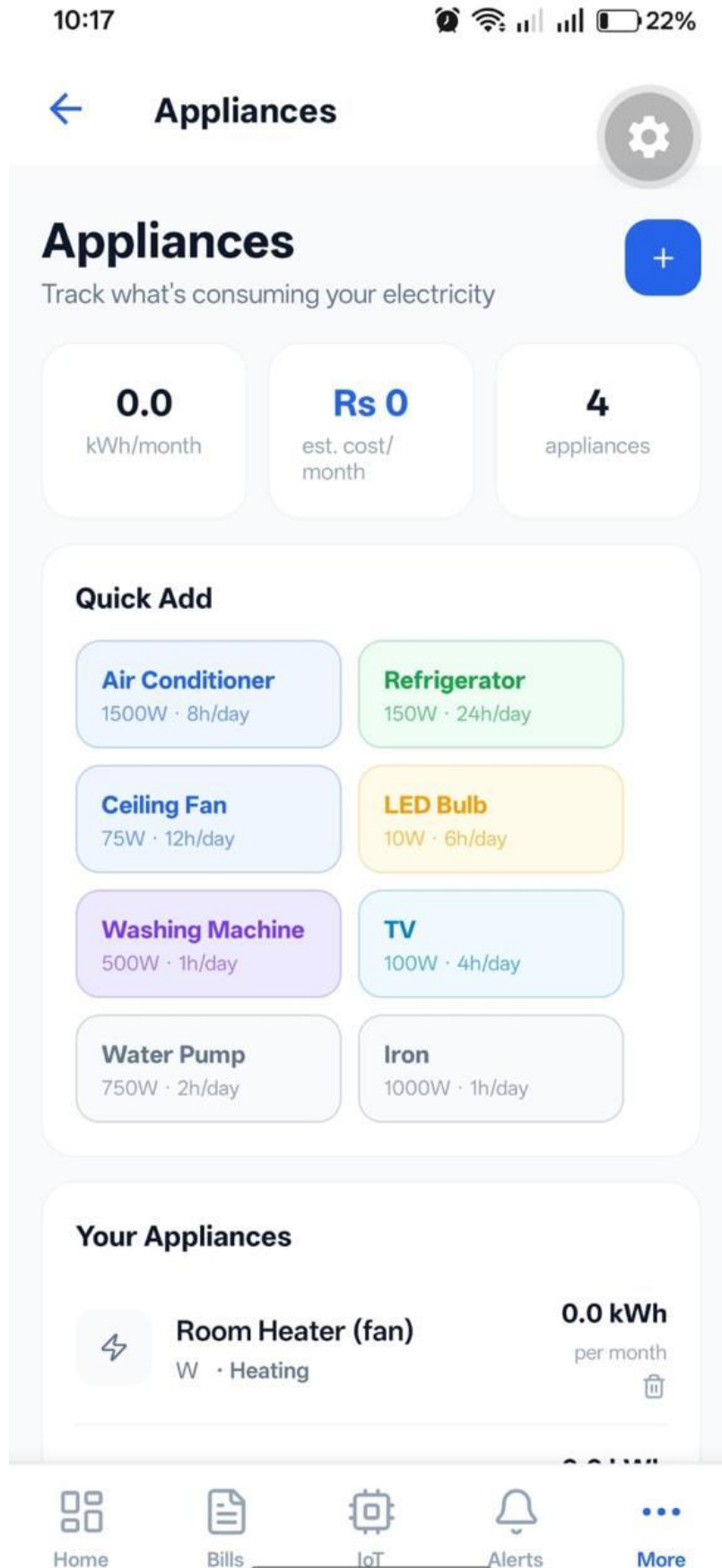
The screens that take input from or give output to the user are: login and registration, the bill upload screen, the dashboard, the prediction view, the budget setup screen, the appliance list, and the recommendations screen. The interface follows simple HCI norms, with clear labels, feedback after every action, and a guided upload flow so a non-technical user can get through it without help.

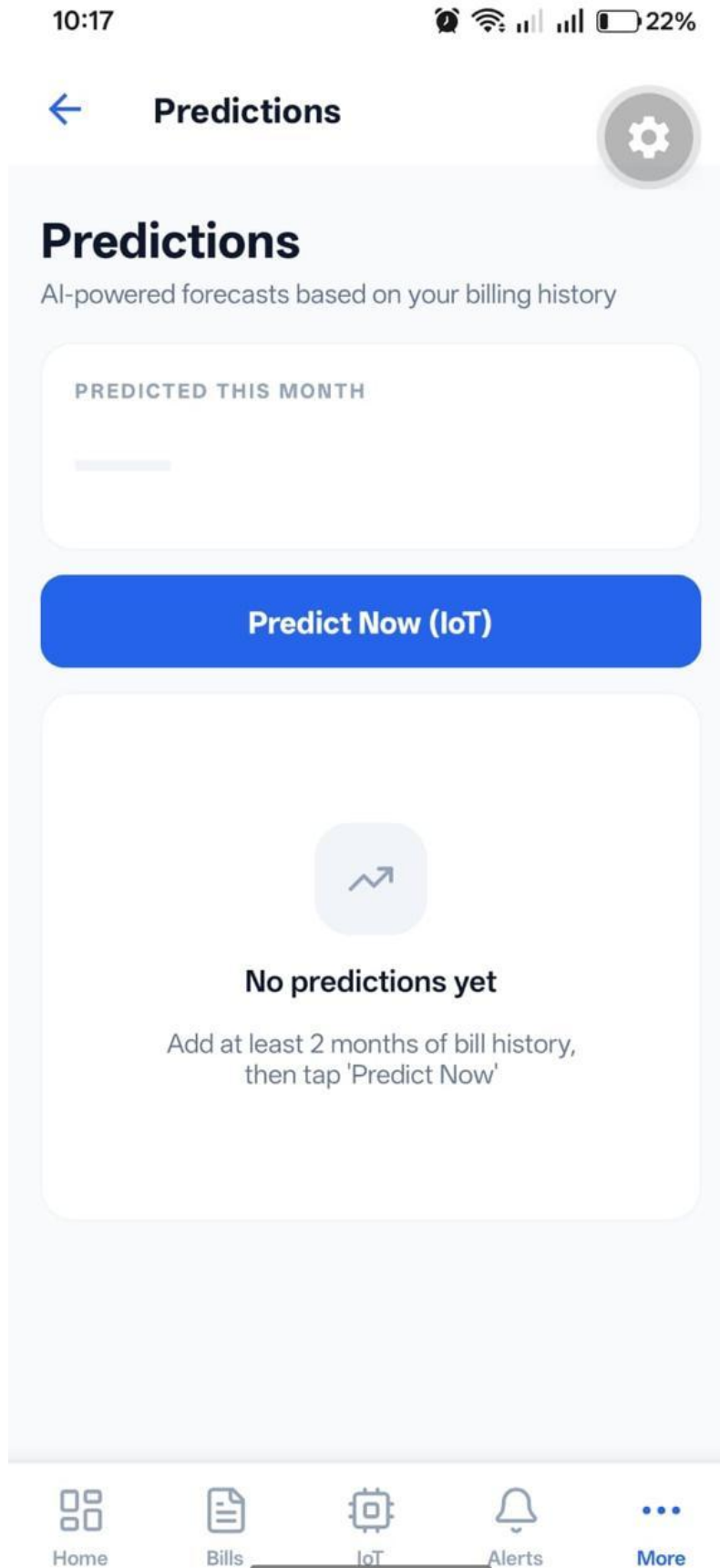
User Screens

- Login/Signup Screen
- Dashboard Screen
- Bill Upload Screen
- Appliance Input Screen
- Budget Management Screen
- Recommendations Screen

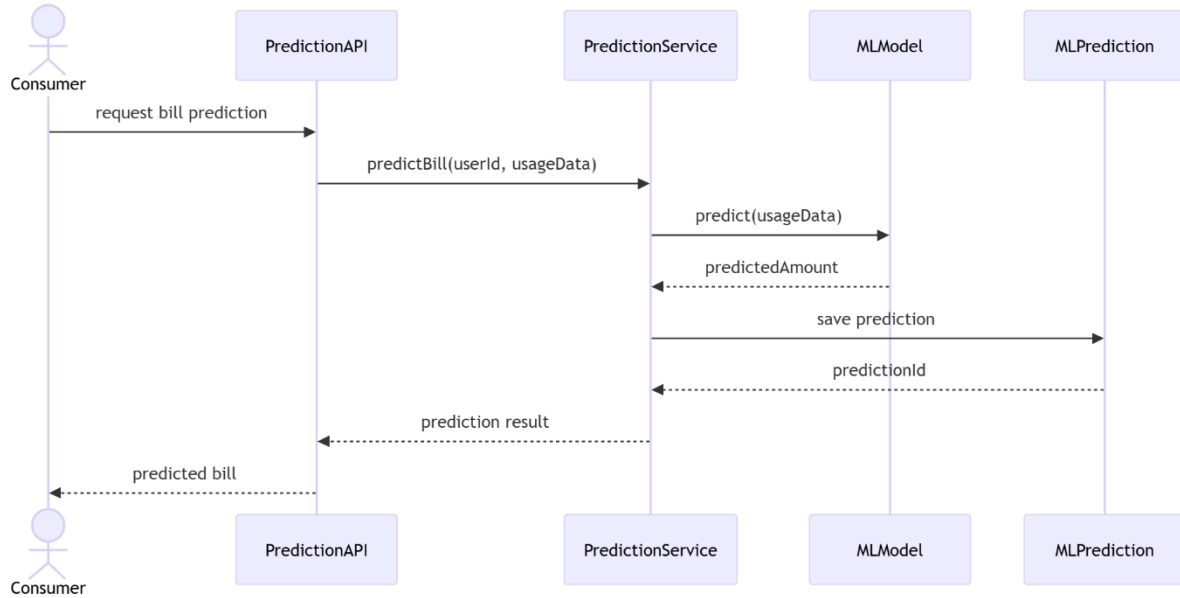




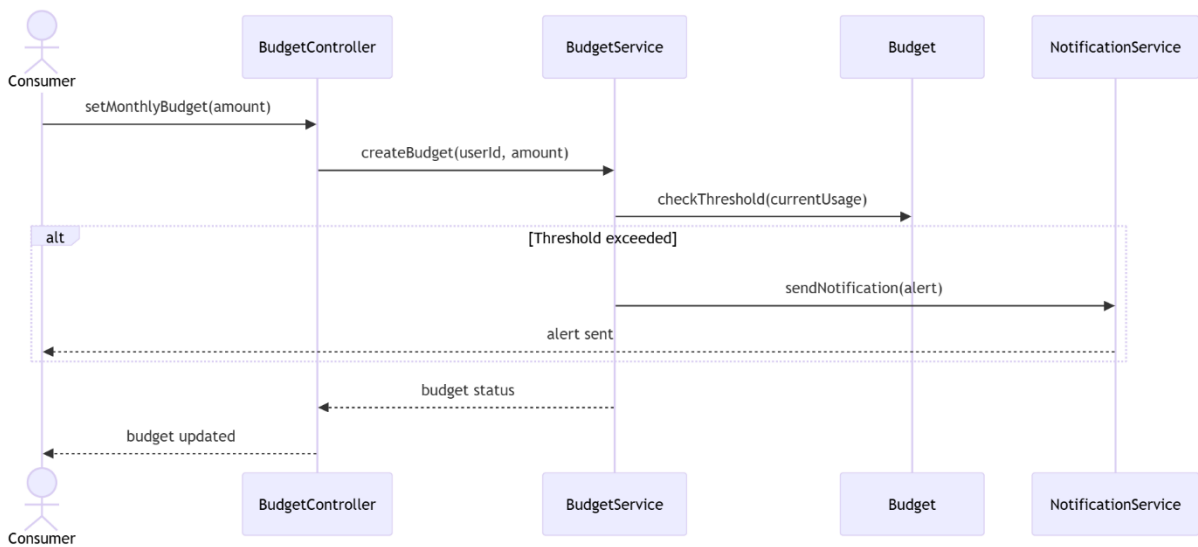




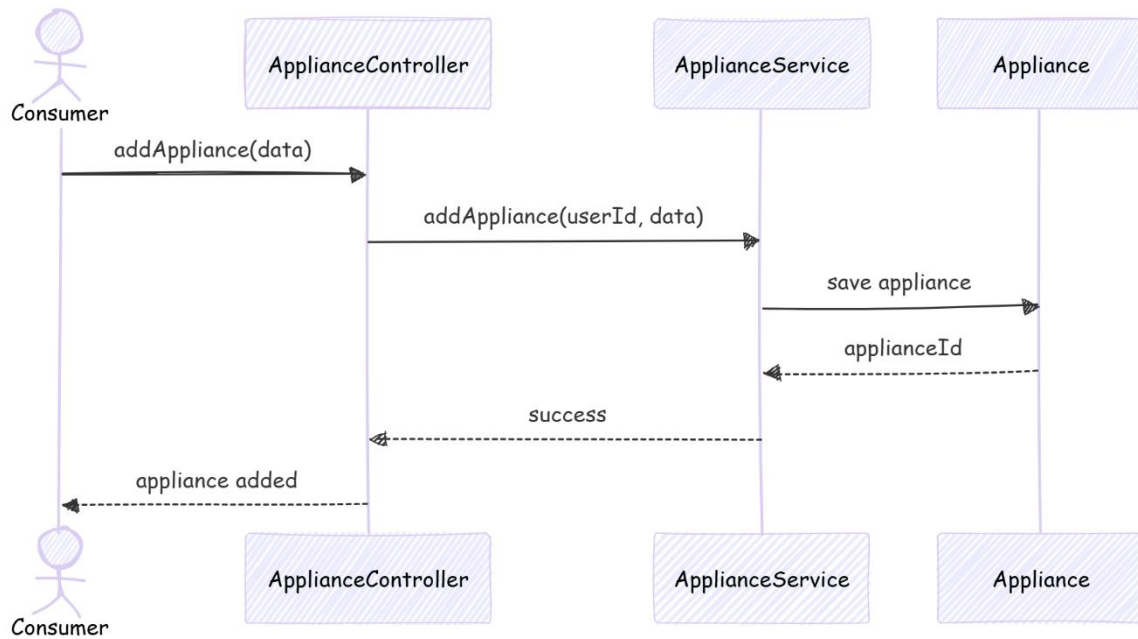
Sequence Diagram Sequence (Get Bill Prediction)



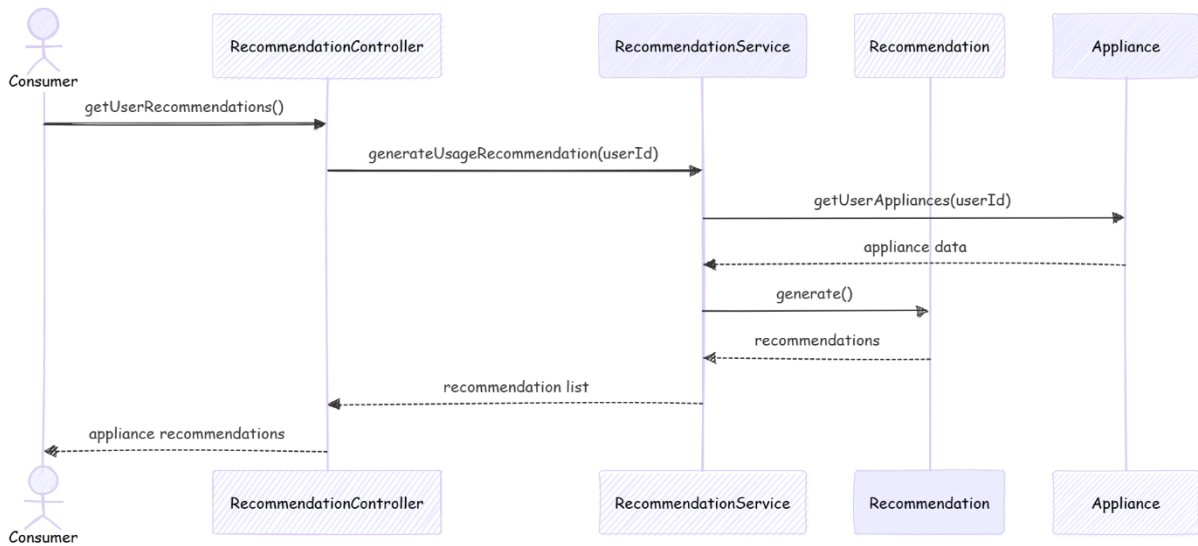
Sequence (Manage Budget Alerts)



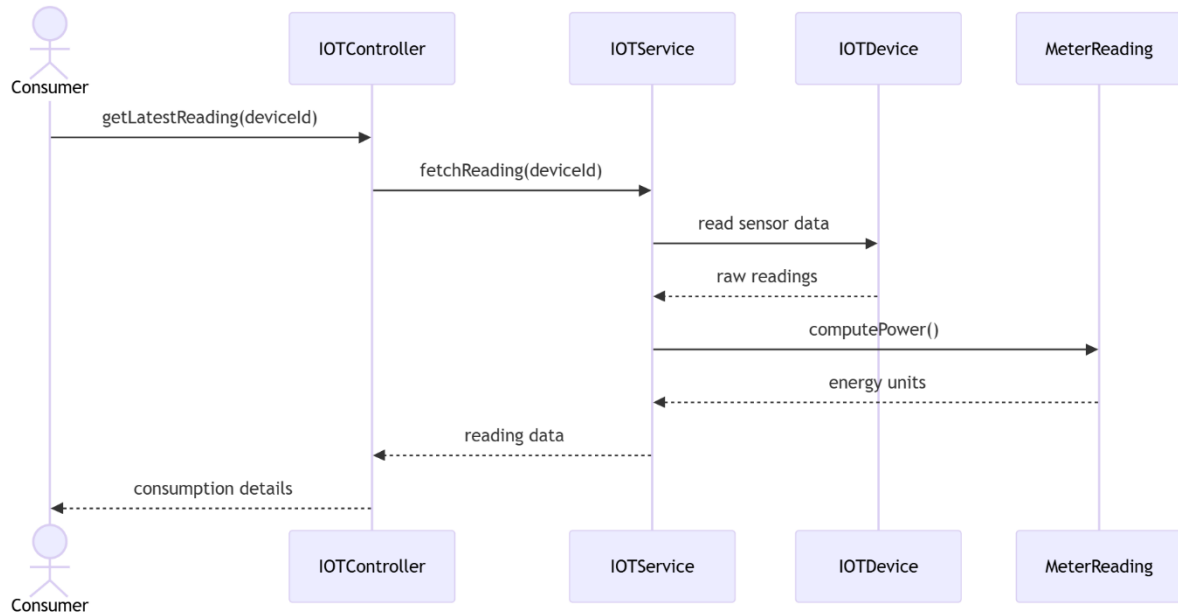
Sequence (Add Appliance Information)



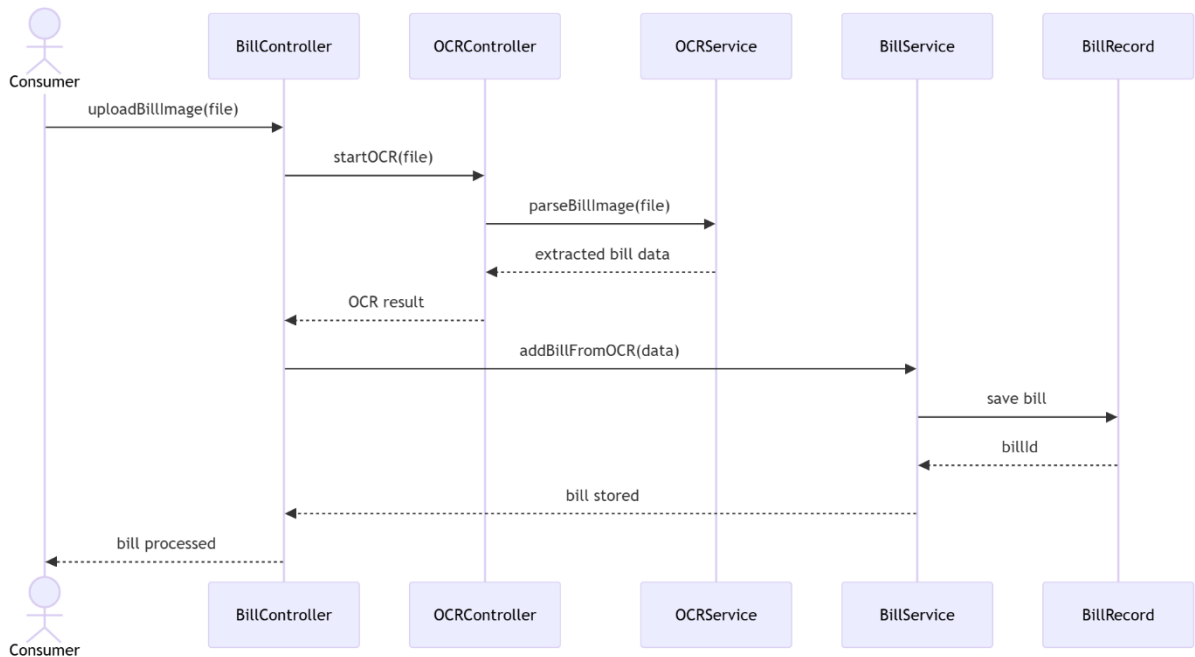
Sequence (Get Appliance Recommendation)



Sequence (Monitor Energy Consumption)



Sequence (Process Electricity Bill)



5.4 Screenshots/Prototype

Here are the complete Design Principles and workflows used for the development of SEBPS.

5.4.1 Workflow

The overall workflow runs as follows. The user registers and logs in. They upload an electricity bill, and the OCR module reads the consumer ID and the bill details. The system then pulls the past billing history for that consumer and stores it. When the user asks for a prediction, the model takes the history, the recent IoT readings, and the appliance profiles and forecasts the units, and the tariff calculator turns those units into an estimated amount. In parallel, the IoT meter keeps sending live readings that show up on the dashboard. The user can set a budget, and the system warns them as the projected cost gets close to the limit, along with appliance level tips on where to cut back.

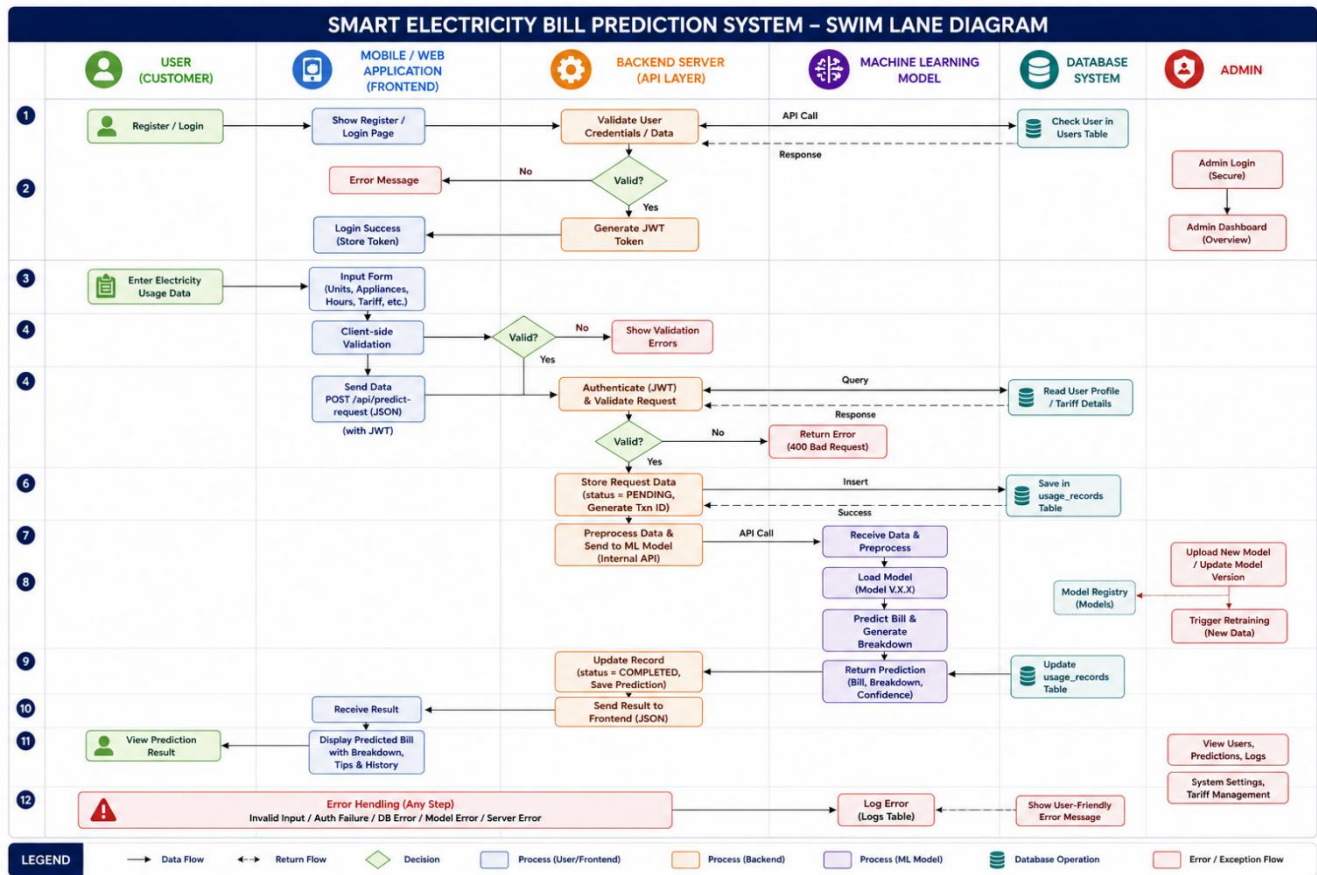
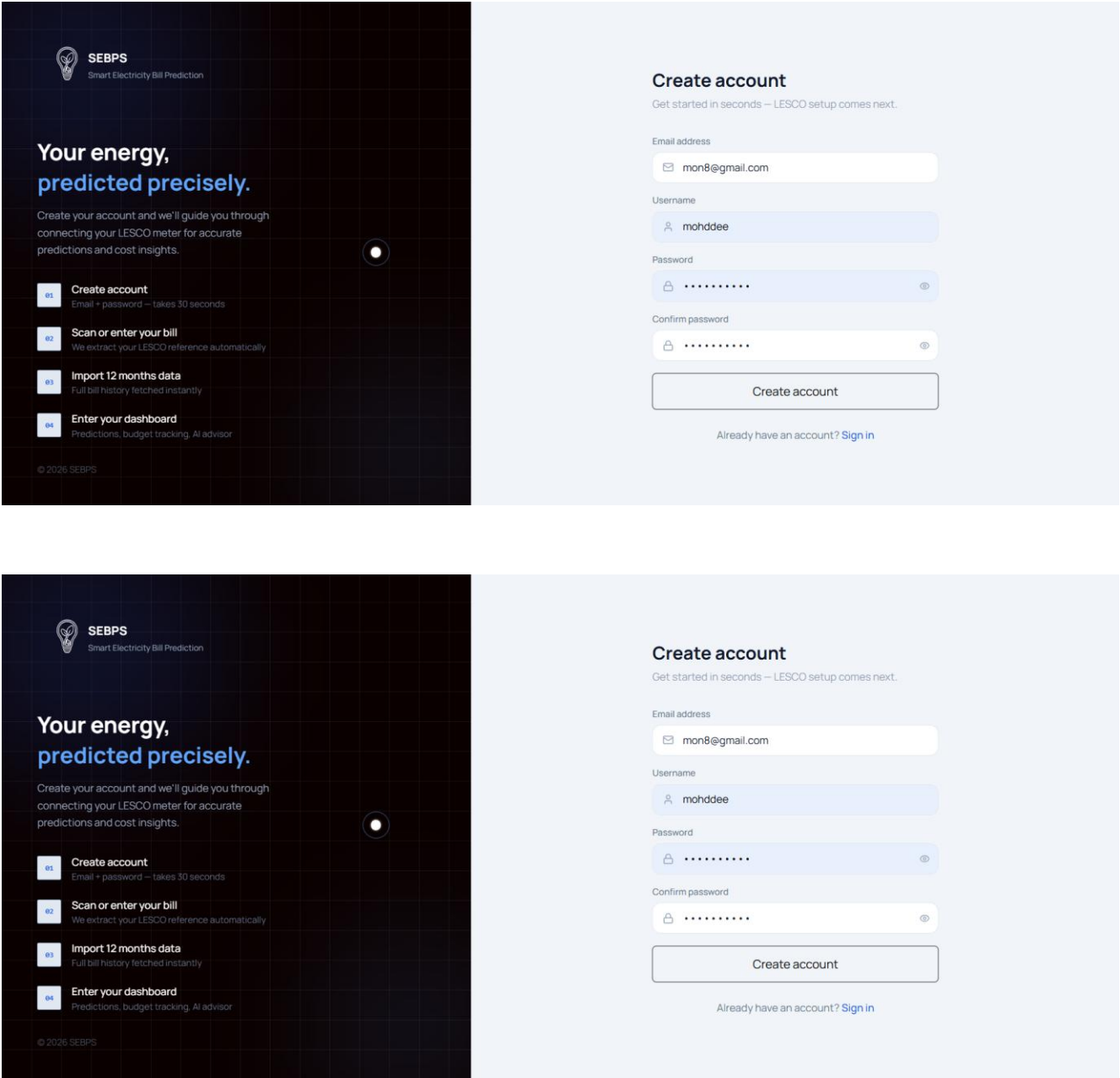


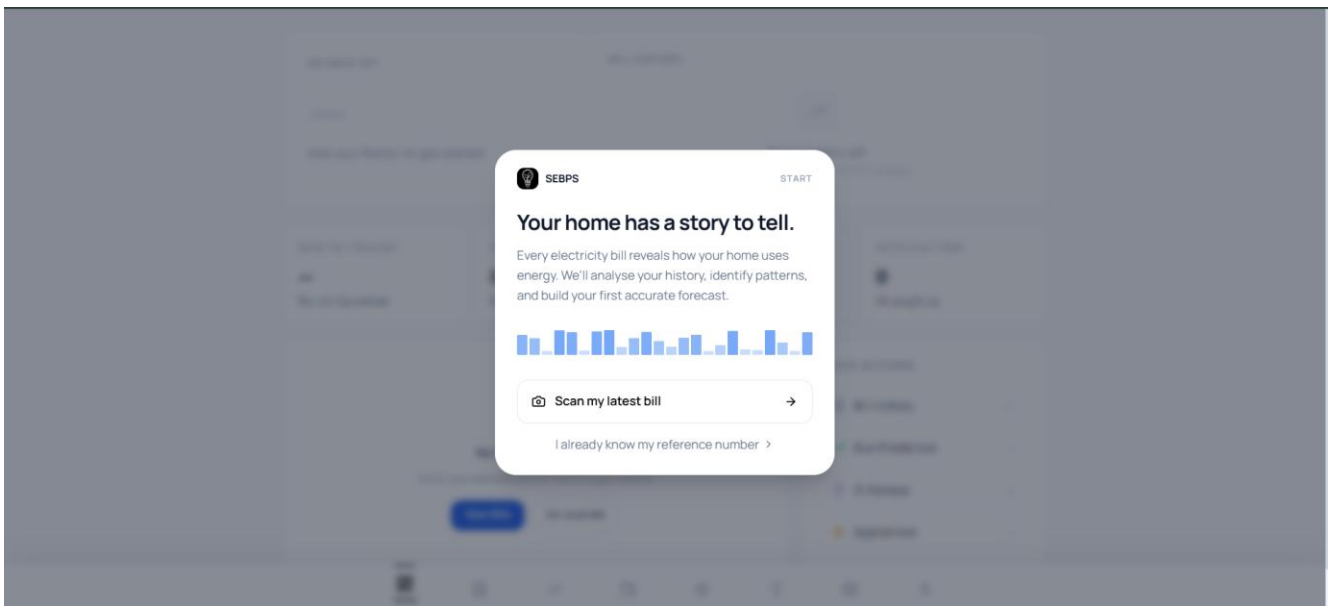
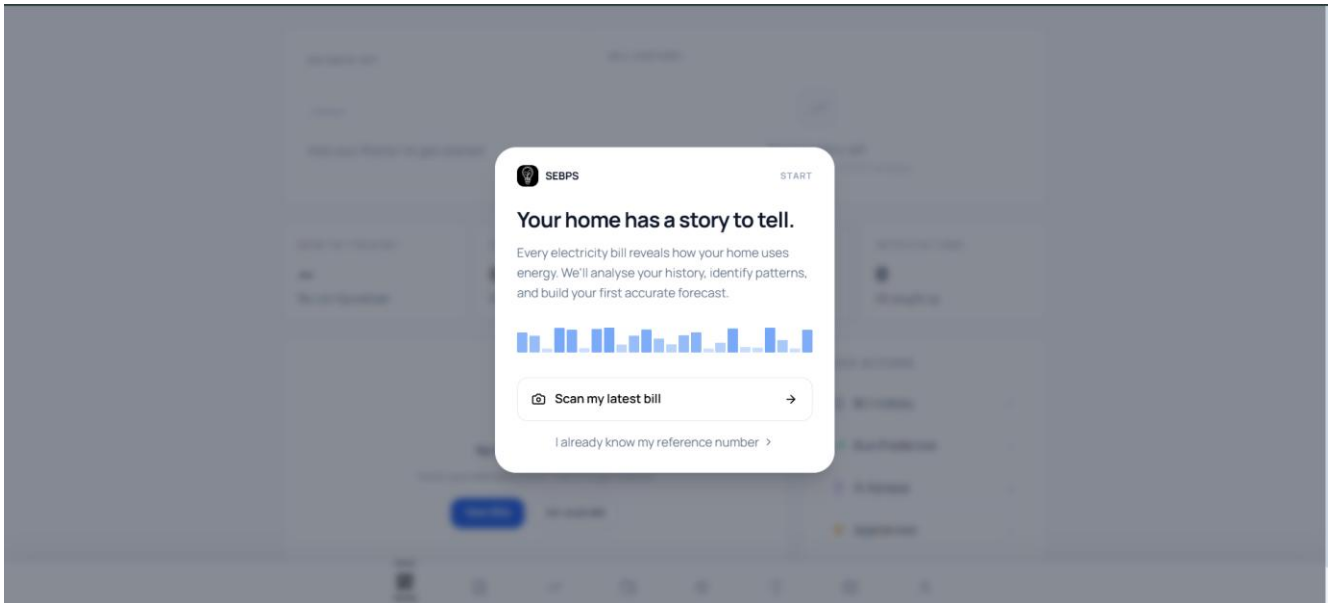
Figure 10: WorkFlow Diagram

5.4.2 Screens

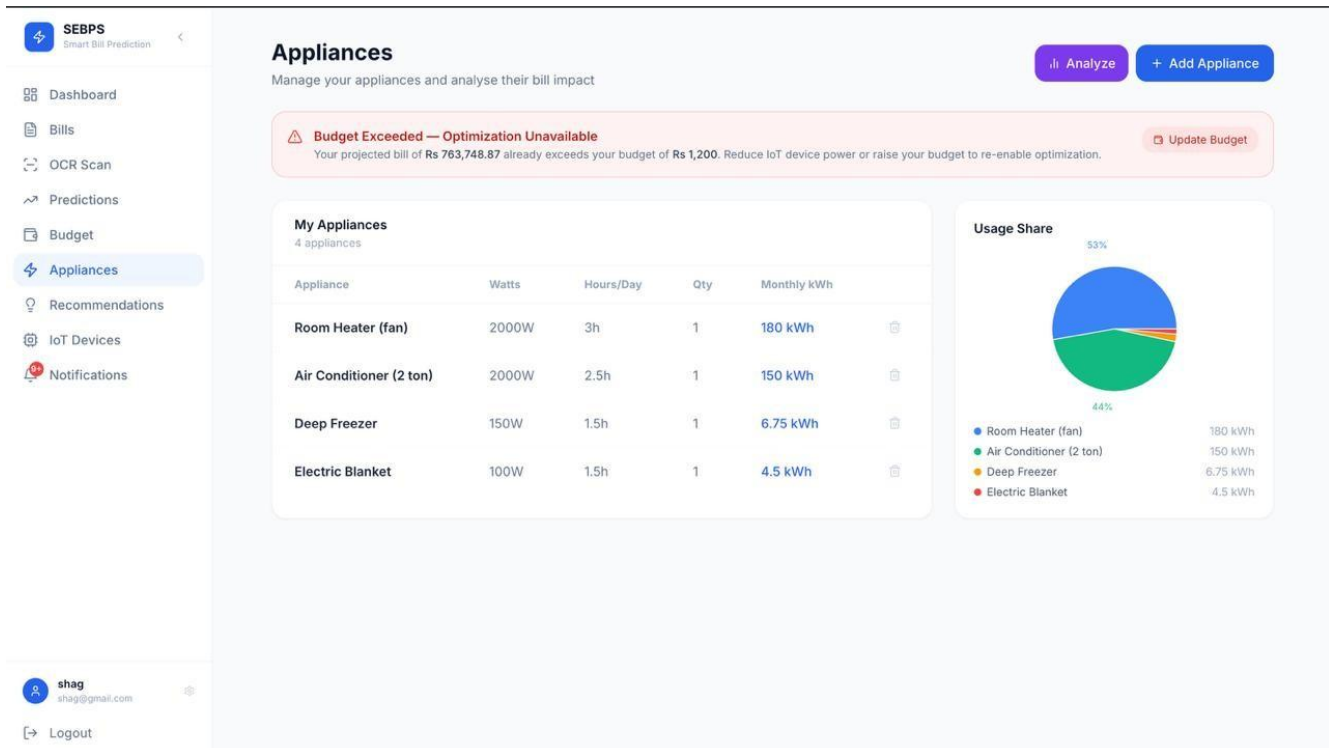
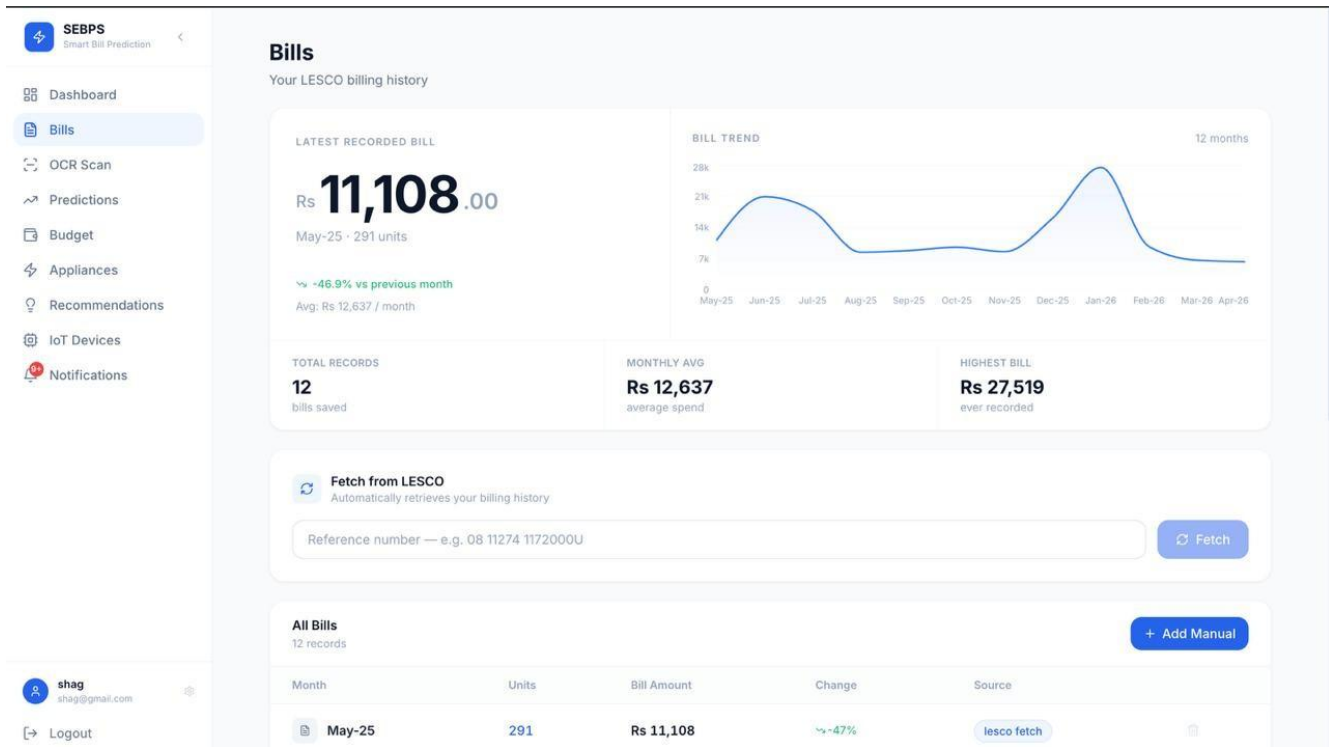
Web Screenshots:

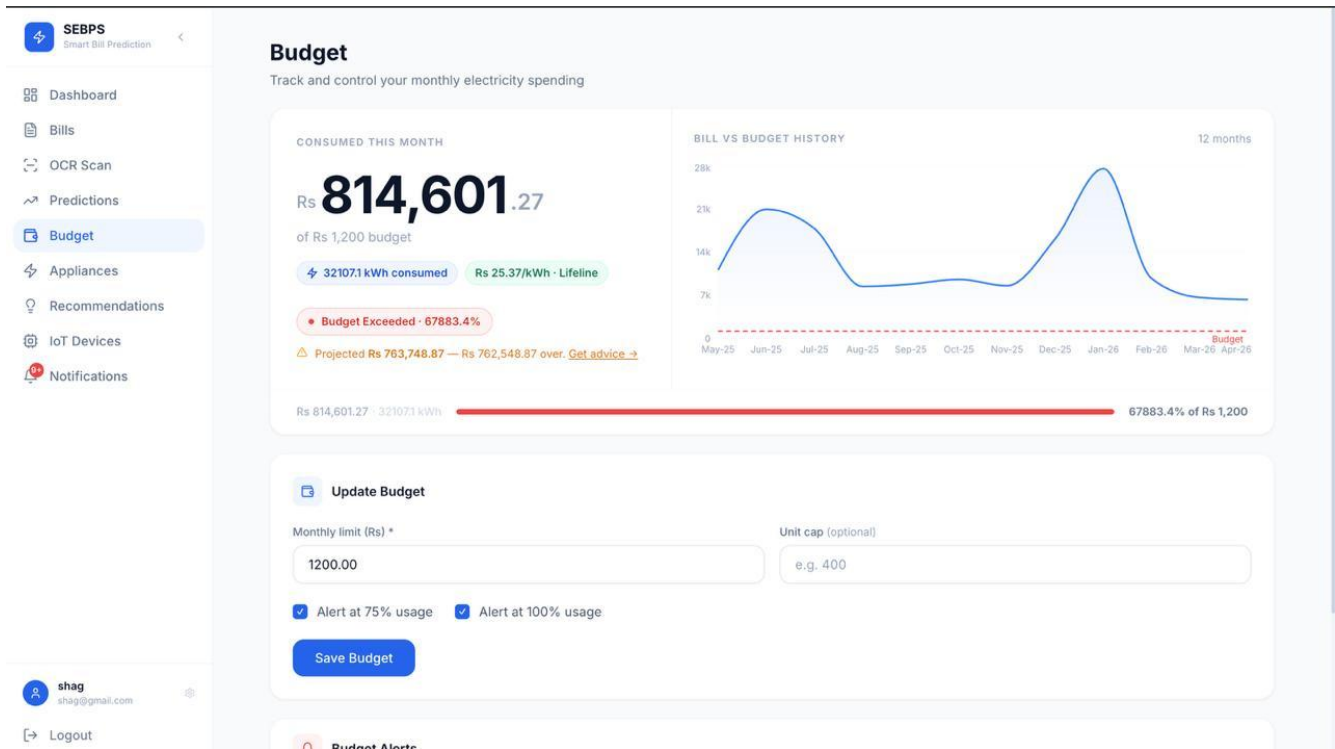


Smart Electricity Bill Prediction System (SEBPS)

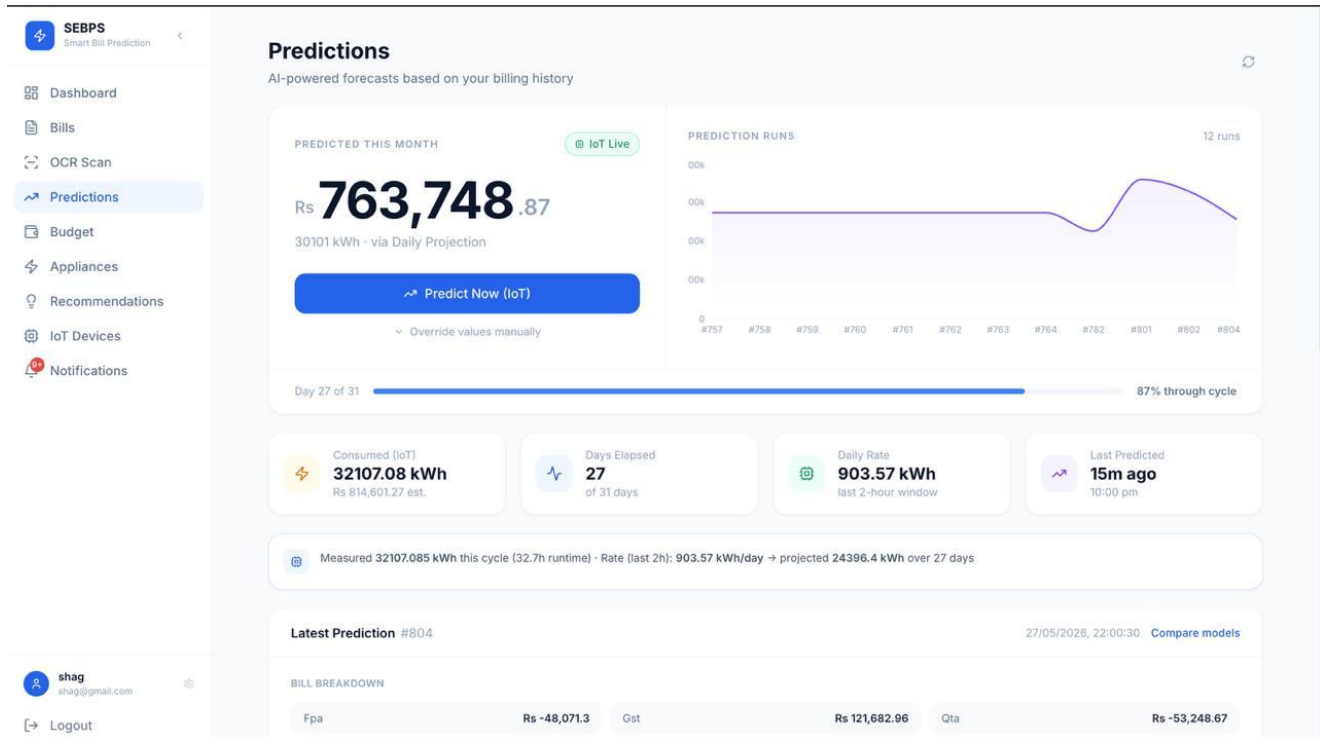


Smart Electricity Bill Prediction System (SEBPS)

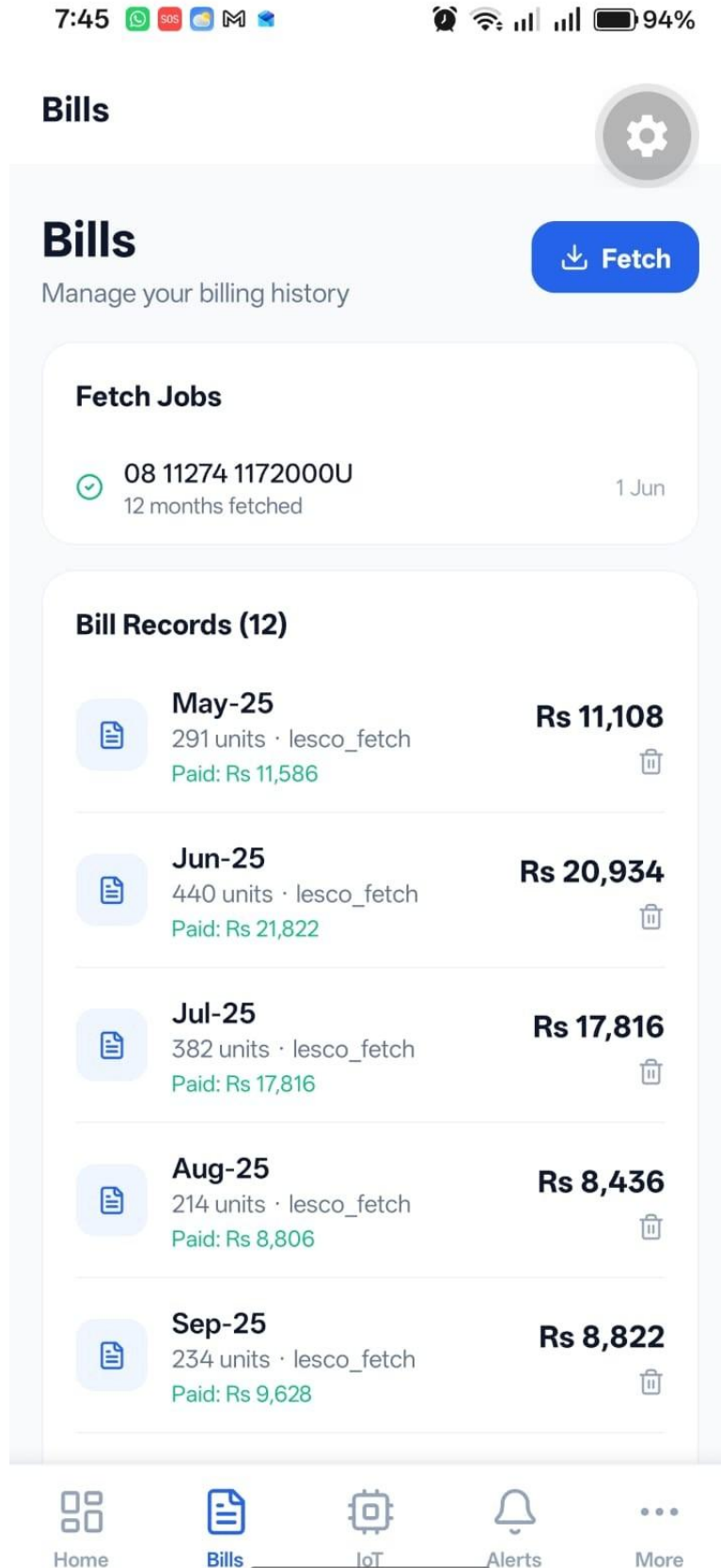




Smart Electricity Bill Prediction System (SEBPS)



Mobile Screenshots:





Recommendations



Smart AI Advisor

Groq AI + IoT-aware budget analysis



On Track

0% used

Rs 0

consumed (IoT)

Rs 0

remaining

Rs 10,520

projected



AI ADVISOR (30 DAYS LEFT)

To stay on track and save more, consider the following two habits:

* **Monitor your daily energy consumption***: Since your average consumption is 274 units/month, try to keep your daily usage in check. Aim to reduce your peak consumption, especially during the summer months when your usage tends to be higher (e.g., 484 units in Jan-26). Reducing your consumption by 50-100 units can help you drop to a lower slab rate, resulting in significant savings.

* **Be mindful of the slab boundaries***: As you're currently predicted to consume 275 units, which falls in the 201-300 slab (Rs 33.10/kWh), try to reduce your consumption to below 200 units to take advantage of the lower slab rate (Rs 28.91/kWh). By doing so,



Home



Bills



IoT



Alerts



More

7:46 

     94%

 Budget



Budget

Track spending against your monthly limit

CONSUMED THIS MONTH

Rs **10,520.09**

of Rs 5,000 budget · via AI prediction



 **Budget Exceeded**

 Projected **Rs 10,520.09** — Rs 5,520.09 over budget

[Edit Budget](#)

[Delete](#)

Budget saved!

LESCO slab tariff

Rs 3.95/unit (≤ 50 lifeline) · Rs 7.74 (51–100) · Rs 22.44 (unprotected 1–100) · Rs 33.10 (201–300) · Rs 47.20 (above 700). Rate recalculates automatically as your IoT meter records more kWh.



Home



Bills



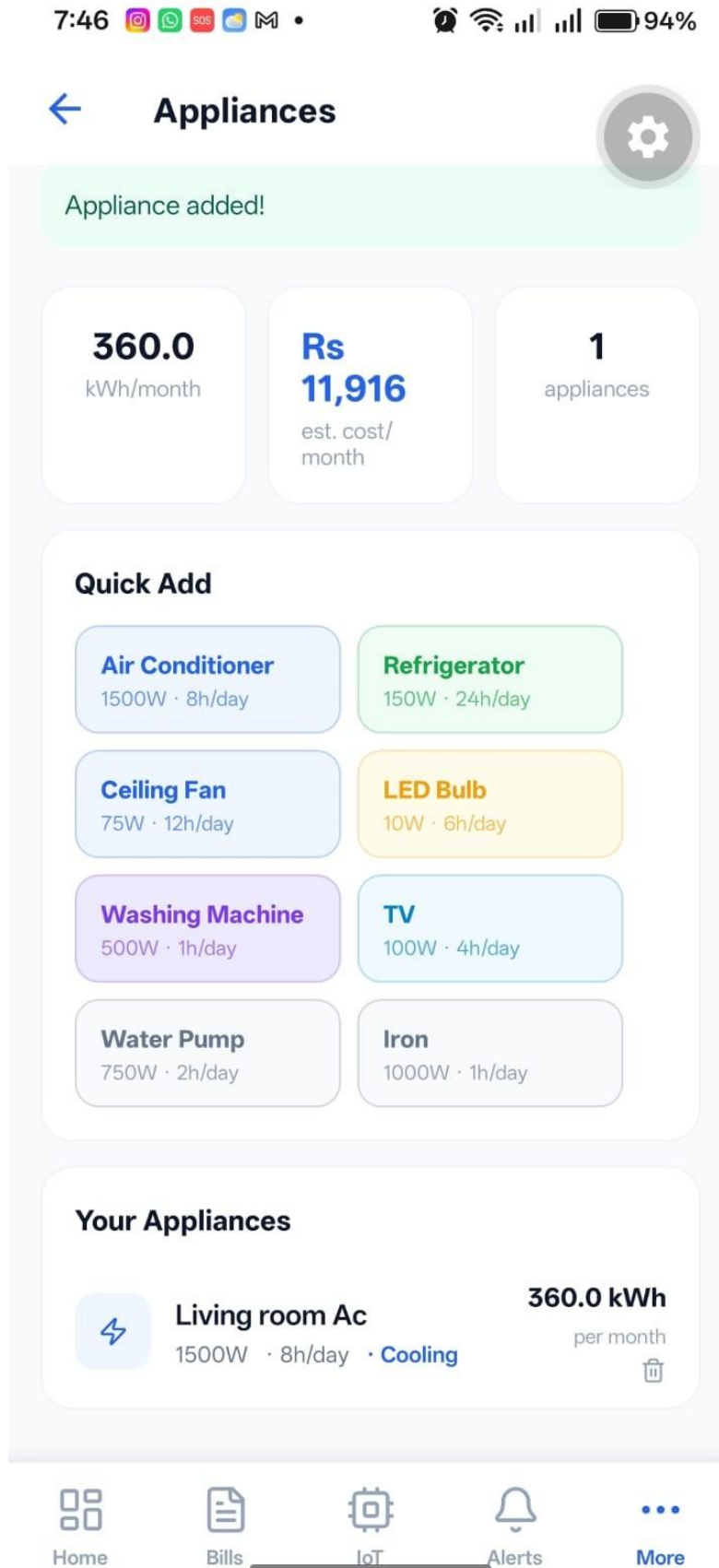
IoT



Alerts



More



5.5 Additional Information

The Smart Electricity Bill Prediction System is designed with flexibility and future expansion in mind. Although the current implementation focuses on LESCO billing rules, the system architecture allows support for other DISCOs in the future with minimal changes. The project is being developed in modular form so that hardware monitoring, bill prediction logic, and the user interface can operate independently. This also helps in testing and debugging individual components. The system is intended for educational and research purposes and does not interact directly with official billing servers.

5.6 Other Design Details

5.6.1 Hardware Interface Design

The hardware setup includes an ESP32 microcontroller connected to a PZEM-004 energy meter module using jumper wires and a breadboard. The PZEM module measures voltage, current, power, and energy consumption in units. The ESP32 reads this data and sends it wirelessly to the backend server using Wi-Fi. Stable power supply and proper wiring are essential to avoid incorrect readings.

5.6.2 Software and Hardware Integration

Real-time data received from the ESP32 is processed on the backend and stored in the database. The frontend dashboard continuously updates live unit consumption and estimated cost. One challenge is maintaining synchronization between hardware readings and web updates, which is handled through efficient API calls and periodic data refresh.

5.6.3 Billing Logic Design

The bill prediction logic is based on appliance load calculation and real-time units. Tariff slabs, fuel adjustment charges, and applicable taxes are applied step by step to produce a final estimated bill that closely matches the official electricity bill.

Chapter 6. Test Specification and Results

6.1 Test Case Specification

Identifier	TC-1: Upload and Extract Bill Data	
Purpose	Allow users to upload electricity bill images and automatically extract consumer ID and billing information using OCR	
Priority	High	
Pre-conditions	User has a valid electricity bill image (LESCO format)	
Post-conditions	Consumer ID and relevant billing data extracted and stored in database	
Typical Course of Action		
S#	Actor Action	System Response
1	User uploads bill image via web/mobile interface	System validates image format and quality
2	User confirms upload	OCR module processes image and extracts consumer ID, billing date, units consumed
3	-	System displays extracted data for user verification
4	User verifies/corrects data	System saves data to database and links to user account
Alternate Course of Action		
S#	Actor Action	System Response
1	OCR fails to extract data	System prompts user to manually enter consumer ID
2	Image quality is poor	System suggests retaking photo with better lighting

Table 12: TC-1: Upload and Extract Bill Data

Identifier	TC-2: Fetch Historical Billing Data	
Purpose	Automatically retrieve one year of billing history from LESCO portal using extracted consumer ID	
Priority	High	
Pre-conditions	Consumer ID has been extracted/entered and is valid	
Post-conditions	12 months of historical billing data stored in database	
Typical Course of Action		
S#	Actor Action	System Response
1	System receives consumer ID	Automated scraper initiates connection to LESCO portal
2	-	System retrieves billing history for past 12 months
3	-	System parses and stores data (units, amount, tariff details)
4	User verifies/corrects data	System displays billing trends and consumption patterns
Alternate Course of Action		
S#	Actor Action	System Response
1	LESCO portal is down	System queues request and retries after 30 minutes
2	Consumer ID not found	System notifies user to verify ID and allows manual entry
3		
...		

Table 13: TC-2: Fetch Historical Billing Data

Identifier	TC-3: Predict Future Electricity Bill	
Purpose	Generate accurate bill forecast using ML model based on historical data and current consumption patterns	
Priority	High	
Pre-conditions	Historical billing data available, ML model trained	
Post-conditions	Predicted bill amount displayed with breakdown of charges	
Typical Course of Action		
S#	Actor Action	System Response
1	User requests bill prediction	System retrieves historical consumption data
2	-	ML model analyzes patterns and

		predicts next month's units
3	-	System applies current tariff slabs, fuel adjustment, taxes
4	-	System displays predicted amount with detailed breakdown
5	User views forecast	System shows confidence interval and factors affecting prediction
Alternate Course of Action		
S#	Actor Action	System Response
1	Insufficient historical data	System uses regional average and notifies user of lower accuracy
2	User changes appliance settings	System recalculates prediction based on new inputs
3		
...		

Table 14: TC-3: Predict Future Electricity Bill

Identifier	TC-4: Monitor Real-Time Consumption Future Electricity Bill	
Purpose	Track live electricity usage through IoT hardware (ESP32 + PZEM-004) and display on dashboard accurate bill forecast using ML model based on historical data and current consumption patterns	
Priority	High	
Pre-conditions	IoT hardware installed and connected to WiFi, linked to user account	
Post-conditions	Real-time consumption data displayed and logged in database	
Typical Course of Action		
S#	Actor Action	System Response
1	Appliances consume electricity	PZEM-004 measures voltage, current, power, energy
2	-	ESP32 processes data and transmits via WiFi every 5 seconds
3	-	Backend receives data and updates database
4	User opens dashboard	System displays live units consumed, current cost, and power usage

5		System updates graphs and trends in real-time
Alternate Course of Action		
S#	Actor Action	System Response
1	WiFi connection lost	ESP32 stores data locally and syncs when reconnected
2	Hardware malfunction detected	System alerts user and suggests troubleshooting steps
3		

Table 15:TC-4: Monitor Real-Time Consumption

Identifier		TC-5: Set Budget and Receive Alerts	
Purpose		Allow users to set monthly electricity budget and receive real-time alerts when approaching limit	
Priority		Medium	
Pre-conditions		User account active, real-time monitoring enabled	
Post-conditions		Budget saved, alert system activated	
Typical Course of Action			
S#	Actor Action		System Response
1	User sets monthly budget limit (e.g., PKR 5000)		System saves budget preference to user profile
2	-		System continuously monitors consumption vs budget
3	-		When 75% budget reached, system sends warning notification
4	-		When 90% reached, system sends urgent alert with reduction tips
5	User views budget status		System shows remaining budget, projected overage, days left
Alternate Course of Action			
S#	Actor Action		System Response
1	Budget exceeded		System suggests specific appliances to reduce usage
2	User disables alerts		System stops notifications but continues tracking
3			

...		
-----	--	--

Table 16: TC-5: Set Budget and Receive Alerts

Identifier		TC-6: Get Appliance-Level Recommendations	
Purpose		Provide personalized energy-saving recommendations based on appliance usage patterns	
Priority		Medium	
Pre-conditions		User has entered appliance details (wattage, usage hours)	
Post-conditions		Customized recommendations displayed to optimize energy consumption	
Typical Course of Action			
S#	Actor Action		System Response
1	User adds appliances with wattage and daily usage		System calculates individual appliance cost contribution
2	-		AI analyzes usage patterns and identifies inefficiencies
3	User views recommendations		System suggests: 'Reduce AC usage by 2hrs/day to save PKR 800'
4	-		System shows alternative usage schedules for cost optimization
5	User applies recommendation		System tracks savings and updates dashboard
Alternate Course of Action			
S#	Actor Action		System Response
1	No appliances added		System provides generic energy-saving tips
2	User rejects recommendations		System learns preference and adjusts future suggestions
3			
...			

Table 17: TC-6: Get Appliance-Level Recommendations

6.2 Summary of Test Results

The one open defect in the prediction module relates to lower accuracy for users with very little history, which is a known limitation rather than a code fault, and is handled by showing a confidence note and leaning on the slab calculator.

Module Name	Test cases run	Number of defects found	Number of defects corrected so far	Number of defects still need to be corrected
OCR Module	TC-1	2	2	0
Billing/History Module	TC-2,TC-8	2	2	0
Prediction Module	TC-3	3	2	1
IoT Monitoring Module	TC-4	2	2	0
Budget Module	TC-5	1	1	0
Recommendation Module	TC-6	1	1	0
Auth/Security	TC-7	1	1	0
AI Chatbot Module	TC-9	2	2	0
Complete System	9	14	13	1

Table 18: Summary of All Test Results

Chapter 7. Project Completion Status/Conclusion

Module Name	Status (Complete, Partially Implemented, Not Implemented)
OCR Module	Complete
Historical Data Retrieval	Complete
Prediction Module	Complete
IoT Monitoring Module	Partially Implemented
Budget and Alerts Module	Completed
Recommendation Module	Complete
AI Chatbot Module	Complete
Web Dashboard	Complete
Mobile View	Complete
Complete System	Complete

Table 19: Project Completion Status

The IoT Monitoring Module is marked partially implemented because the core work was done. The main hardware part has been crafted to the best of our knowledge. However, some portion of the Hardware is still in progress.

Target/Objective	Status (Completed, Partially Completed, Not Completed)	Reason(s)
Predict monthly bills from historical and real time data	Completed	Per user models and the slab calculator produce estimates within an acceptable range
Use OCR to remove manual entry	Completed	EasyOCR reads clear bills reliably, with manual entry as a fallback

Train ML models with acceptable accuracy	Completed	Accuracy is good for users with enough history; new users get lower confidence forecasts
Real time monitoring through IoT	Partially Completed	The ESP32 and PZEM-004 unit streams live readings to the dashboard
Budgeting and saving recommendations	Completed	Budgets, threshold alerts, and appliance tips all work
Secure, easy to use web and mobile interface	Completed	Web interface is complete; the mobile view covers the core screens
Number of Targets Completed	5	Done
Number of Targets Partially Completed	1	Uncertain Government Conditions.
Number of Targets Not Completed	0	None

Table 20: Objective(s)/Target(s) Status

References

1. National Electric Power Regulatory Authority (NEPRA), Schedule of Electricity Tariff, S.R.O No. 279(I)/2026, Government of Pakistan, 2026.
2. Lahore Electric Supply Company (LESCO), Consumer Bill Information Portal, <https://www.lesco.gov.pk>
3. EasyOCR, Ready to use OCR library, JaidevAI, <https://github.com/JaidevAI/EasyOCR>
4. OpenCV, Open Source Computer Vision Library, <https://opencv.org>
5. scikit-learn, Machine Learning in Python, Pedregosa et al., Journal of Machine Learning Research, vol. 12, pp. 2825 to 2830, 2011, <https://scikit-learn.org>
6. Espressif Systems, ESP32 Technical Reference Manual, <https://www.espressif.com>
7. PZEM-004T Energy Meter Module, Peacefair, Datasheet and Communication Protocol.
8. Express.js, Node.js web application framework, <https://expressjs.com>
9. React, A JavaScript library for building user interfaces, <https://react.dev>
10. Recharts, Composable charting library for React, <https://recharts.org>

Appendix A Glossary

SEBPS

Smart Electricity Bill Prediction System. The proposed system that estimates electricity bills and monitors real-time energy consumption.

LESCO

Lahore Electric Supply Company. The electricity distribution company whose billing structure is used in this project.

ESP32

A low-cost microcontroller with built-in Wi-Fi capability used to transmit real-time electricity consumption data to the server.

PZEM-004

An energy meter module used to measure voltage, current, power, and energy units consumed by household appliances.

Energy Units (kWh)

Kilowatt-hour. The standard unit of electrical energy used for electricity billing.

Tariff Slabs

Predefined ranges of electricity consumption set by the government, where unit price changes based on total usage.

Fuel Adjustment Charges

Additional charges applied to electricity bills based on variations in fuel costs used for power generation.

Real-Time Monitoring

The continuous measurement and transmission of electricity consumption data as it is being used.

Frontend

The user interface of the system that allows users to view consumption data, predicted bills, and system outputs.

Backend

The server-side component responsible for data processing, calculations, and communication between hardware and frontend.

Database

A storage system used to save user information, appliance details, and energy consumption records.

Wi-Fi Connectivity

Wireless network connection used by the ESP32 to send data to the backend server.

Web Application

A browser-based application that users access to interact with the system

Appendix B IV & V Report

(Independent verification & validation)

IV & V Resource

NameSignature

S#	Defect Description	Origin Stage	Status	Fix Time	
				Hours	Minutes
1					
2					
3					
...					

Table B.21: List of non-trivial defects

Appendix C Deployment/Installation Guide

SEBPS can be used in two ways: through the live hosted web app, or by running the full stack locally with Docker. The hosted version is the easiest for an evaluator, while the local setup is for development and for running every service on your own machine.

Using the Hosted Web App

The web application is deployed on AWS and is available at:

<https://sebps.work.gd/>

A demo account is provided for testing:

- Email: shag@gmail.com
- Password: shag@1234

Just open the link in any modern browser and log in. No installation is needed for this path.

Running the Full Stack Locally with Docker

The whole system runs as a single Docker Compose stack made up of seven services, so the entire app can be brought up with one command.

Prerequisites

- Docker Desktop installed and running
- Expo Go installed on your phone, on the same Wi-Fi network as your machine (only needed if you want the mobile app)

Step 1: Configure the environment

Copy the example environment file:

```
cp .env.example .env
```

Open .env and set your machine's LAN IP, which is the IP your phone can reach. To find it:

```
# macOS
```

```
ipconfig getifaddr en0
```

```
# Linux
```

```
hostname -I | awk '{print $1}'
```

Then set it in .env:

```
# .env
```

```
HOST_IP=192.168.1.52 (replace with your actual LAN IP)
```

```
F25CS132
```

Create the backend secrets file and fill in the keys:

```
cp backend/.env.example backend/.env
# Fill in GROQ_API_KEY and the LESCO credentials in backend/.env
```

Step 2: Start everything

```
docker compose up --build
```

The first run takes around five minutes, since it downloads the images, installs the dependencies, and loads the ML models. Once it is up, the services are reachable at:

Service	URL
Web app	http://localhost:5173
API docs	http://localhost:8000/api/v1/docs/
Mobile	Open Expo Go, choose Enter URL manually, then <code>exp://HOST_IP:8081</code>

A QR code for the mobile app also appears in the mobile container logs:

```
docker logs lesco_fyp-mobile-1
```

Step 3: Later runs

```
docker compose up      # no --build unless dependencies changed
docker compose down    # stop everything (data is kept in volumes)
docker compose down -v # stop and wipe all data for a fresh start
```

Backend only (no frontend or mobile)

```
cd backend
docker compose up --build
```

Service Overview

Container	Role
db	TimescaleDB (PostgreSQL 16) for time series IoT data
redis	Celery broker and result backend
api	Django with Daphne (HTTP and WebSocket)
celery_worker	Async tasks: OCR, LESCO fetch, predictions
celery_beat	Scheduled tasks such as IoT polling
frontend	Vite dev server with an /api proxy
mobile	Expo Metro bundler

Project Structure

|— backend/ Django API (DRF, Channels, Celery, TimescaleDB)

— frontend/	React + Vite web dashboard
— mobile/	Expo React Native app
— module_1_predictor/	Bill prediction model
— module_2_ocr/	EasyOCR + Tesseract bill scanner
— module_3_fetcher/	LESCO history scraper
— module_4_recommender/	Rule based energy tips
— module_5_chatbot/	Groq powered AI advisor
— docker-compose.yml	Single command stack (all 7 services)

OR SIMPLY BROWSE : <https://sebbs.work.gd>

This will take you to the webpage where the application is installed.

Appendix D User Manual

This manual walks a household user through SEBPS from the first login to daily use. The easiest way to start is the hosted web app at <https://sebbs.work.gd/>, which works in any modern browser. A demo account (shag@gmail.com / shag@1234) is available if you just want to look around.

Getting Started

Create an account or log in. Open <https://sebbs.work.gd/>. New users register with an email and password, and returning users log in. After logging in you land on the dashboard, which is the home base for everything else.

Find your way around. The main sections are reached from the side menu: Dashboard, Bills, Predictions, IoT monitoring, Budget, Appliances, Recommendations, Chatbot, Notifications, and Profile. Each one is covered below.

Adding Your Electricity Data

Upload a bill. Go to the Bills or Upload screen and choose a clear photo or scan of your LESCO bill. The OCR scanner reads the consumer ID, the units, and the date. Give the extracted values a quick check and correct anything that looks wrong before you save. If a photo comes out too blurry to read, retake it in better light and try again.

Let the system fetch your history. Once your consumer ID is saved, SEBPS pulls your past billing records from the LESCO portal and shows your consumption trend. This history is what the prediction model learns from, so the more months it can pull, the better the forecast.

Enter your consumer ID by hand if needed. If the portal is down or the OCR cannot read a bill, you can type the consumer ID in directly and continue.

Predicting Your Bill

Open the Predictions screen. SEBPS estimates the units and the cost for your coming month based on your history and, if you have the meter connected, your live usage so far. The result shows a predicted amount with a breakdown of the slab charges, the fuel adjustment charge, and taxes, so you can see how the figure was reached. A confidence note tells you how reliable the estimate is, which will be lower for new users who only have a few months of history.

Live Monitoring with the IoT Meter

Connect the meter. If you have the ESP32 and PZEM-004 unit, power it on and make sure it is on Wi-Fi and linked to your account. The IoT screen and the dashboard then show your live units, current power draw, and running cost, and the graphs update on their own as new readings come in.

Handle short outages. If the meter loses Wi-Fi for a moment, it holds the readings and sends them once it reconnects, so brief drops do not leave gaps.

Budgeting and Alerts

Set a monthly budget. On the Budget screen, enter the amount you want to stay under for the month. SEBPS tracks your projected cost against it. You get a warning when you reach about 75 percent of the budget and a stronger alert at 100 percent, with tips on where to cut back. The budget widget on the dashboard shows how much room you have left.

Appliances and Recommendations

Add your appliances. On the Appliances screen, list your main appliances with their wattage and how many hours a day you use them. SEBPS then works out how much each one adds to your bill.

Read the recommendations. The Recommendations screen turns that into practical advice, for example how much you could save by cutting a heavy appliance's daily use, so you can decide what is worth changing.

The AI Chatbot

Ask in plain words. Open the Chatbot screen and ask anything about your electricity. It is tuned for energy and billing questions and uses your live readings and recent bills, so you can ask things like how many units you have used today or why your bill looks higher than last month, and get an answer based on your own data. If you ask something unrelated to electricity, it will point you back to what it can help with.

Notifications and Profile

Stay informed. The Notifications screen keeps your budget alerts and other messages in one place.

Manage your account. From the Profile screen you can update your details, manage your linked meter, or delete your account and all your data.



University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

Faculty of Information Technology

PROJECT OFFICE

CERTIFICATE OF APPROVAL FORM

DATE

Day		Month		Year	
1	3	—	0	7	—
2	0	2	6		

Project Information:

Group ID:	F25CS132	Advisor's Name:	Mr. Numan Aslam
Thesis / Project Title:	Smart Electricity Bill Prediction System (SEBPS)		

Advisor's Approval:

I Mr. Numan Aslam certify that, I have;

- ☒ Reviewed the document thoroughly.
- ☒ Taken mock presentation and demonstration of project.
- ☒ Verified the smooth working of the project and met the proposed completion requirements.
- ☒ Verified that the CD is properly burned with all phase documents, user manual, deployment guide and installable software along with source code.

Signatures and Date


07/10/26

Advisor

For Project Office use only

Remarks:		
Group No	F25CS132	Manager Projects